

Agent Memory Selection under Bounded Context

Which History Records Matter for Software-Engineering Agents?

Jorge Simão

Draft — September 2026

Abstract

Long-running language-model agents accumulate task statements, model actions, tool observations, source-code views, edits, tests, error messages, and intermediate hypotheses. The common baseline is to replay as much recent history as the model context permits, but recency alone ignores the causal and semantic roles of agent state. This paper studies agent-history selection as a separate problem from inference-engine caching: given a canonical trajectory, which records should remain active for the next model call?

We introduce an engine-independent record abstraction and evaluate bounded memory policies for software-engineering agents. The first harness is mini-swe-agent, whose linear message history, bash-only action interface, and minimal control loop make memory selection directly observable. We concentrate on full history, matched token/causal recency, whole-record state-aware retirement, and payload-only retirement that preserves compact provenance/control evidence. An offline oracle estimates removable-history headroom. We separate record selection from within-record materialization and evaluate both frozen-trajectory counterfactuals and autonomous SWE-bench runs.

The primary outcomes separate final solution-state success from correct agent submission, and report official task resolution, model calls, selected input tokens, repeated source acquisition, repeated verification, and first-divergence behavior. The goal is not to optimize one serving engine, but to derive a portable agent-memory policy that can later be realized as text, persistent native K/V, or other backend state. The evidence challenges the premise that state supersession implies safe memory deletion: operationally obsolete payload can remain useful as trajectory provenance, localization evidence, or control state. In the first autonomous single-task pilot, however, same-version/span read supersession saves 6.43% of cumulative input tokens, changes the trajectory at its first exclusion, and ends in official failure. The primary full-history submission resolves only one of two executions, so this is evidence of a behavioral effect rather than an identified task-success penalty. On a second task identity, two new full-history controls again resolve one of two primary submissions despite identical declared generation settings and exact initial workspace identity. Post-hoc auxiliary grading of the two failed full-history submissions shows that both terminal workspaces contain resolving patches: the failures are in submission protocol rather than solution state. Calls and actions nevertheless remain unstable, so we stop the second task before any heuristic arm. The H3 terminal workspace also fails its auxiliary grade. On a seeded, repeatable frozen trajectory, structured evidence inside oversized tool observations saves 5.54% of cumulative input while preserving 12/22 conservatively equivalent actions; more aggressive matched-span and head/tail controls save roughly four times more on a fixed slice but preserve fewer actions. A first autonomous structured-evidence run resolves officially in 22 calls while saving 6.10% of its own trajectory-counterfactual input; this is a promising single-task point, not an accuracy estimate. Model-visible H1/H2 receipts require an explicit tokenizer-based size gate: short receipts fail closed to canonical bytes, while oversized synthetic controls compress observations by more than 95% and retain valid terminal actions. These receipt rows qualify the mechanism, not autonomous task utility. Repeated same-host qualification on three task identities gives FULL 5/6 and matched-recency 4/6 primary resolutions. Matched recency saves

6.35% of its pooled materialized history tokens, but its 66.7% task-macro resolution is below the predeclared 80% gate. On one task both treatment repeats make the intended source edit and then submit a source excerpt rather than a unified diff. Failure-aware accounting prevents those shorter failed runs from being reported as efficiency gains. In the persistent three-issue cohort, six clean FULL controls resolve 18/18 issues. R4/M2/V2 resolves 4/6, two R6/M2/V2 executions resolve 5/6 with mean failure-aware saving of -1.52% , and R8/M2/V2 resolves 3/3 but saves only 13.09% while adding six calls. A frozen same-prefix Scikit counterfactual localizes the R4 first divergence: FULL yields a valid Bash action in 5/5 repeats, R4 in 0/5, and 24/25 one-group add-backs restore a valid action. The broad repair set implicates tool-protocol conditioning rather than one omitted task fact and motivates a generic clean action-observation exemplar floor. The resulting episode-aware R4/M2/V2/P1 policy resolves 6/6 issues in two autonomous executions, saves 36.41% and 45.82% against contemporaneous FULL, and reduces calls by six and nine. Removing the episode signal changes the result: boundary-free global R4/M2/V2/P1, while preserving every user instruction, resolves only 2/3, increases calls from 55 to 66, and is assigned zero failure-aware saving. A two-level boundary-free instruction- epoch policy instead keeps the active epoch whole and retires only earlier assistant-tool detail. On a clean 3/3 persistent-FULL replication, this E0 policy also resolves 3/3, saves 34.83% against the paired control, and reduces calls from 66 to 64. This is one ordered sequence family, not yet a general deployment profile. At $N=6$ the same aggressive E0 rule resolves 6/6 versus FULL’s 5/6 and removes 55.34% on its own trajectory, but calls rise from 60 to 114; against the paired FULL denominator it consumes 43.87% more input tokens. Per-request retention is therefore not an efficiency result unless trajectory length is included. Retaining one prior epoch (E1) raises resolution from 4/6 to 5/6 in a subsequent ladder but adds eight calls and yields only 14.21% raw paired saving. Retaining two (E2) reduces calls by two and yields 21.93% raw paired saving, but loses one FULL success, so its failure-aware saving is zero. The ladder control also contains a submission error and is ineligible for a positive paired claim. Neither E1 nor E2 advances. A targeted same-prefix diagnosis then reverses the apparent E2 loss: two E2 forks resolve 2/2 with identical 13-call patches, while two FULL forks resolve 0/2 with different incorrect patches. E2 sends 21.92% fewer tokens at equal mean calls, but the reversal is diagnostic evidence of trajectory instability, not a policy qualification result. The resulting thesis is that obsolete state and obsolete agent memory are different concepts; bounded memory should retire payload separately from provenance and control. The central evaluation therefore includes both fresh one-issue sessions and persistent sequences of one to ten issues. A persistent policy is credited only against the appropriate baseline: matching a fresh reset is automatic session hygiene, while an advantage over fresh sessions requires preserved cross-issue utility or reduced rediscovery.

1 Introduction

Software-engineering agents differ from ordinary retrieval-augmented prompts in one crucial way: their context is not a static evidence set. It is a growing history of actions and consequences. A coding trajectory may contain the original task, source-code observations, edits, failed commands, test results, intermediate hypotheses, and final verification. Many of these records become irrelevant quickly, while others remain necessary long after they were created.

The usual history policy is recency. Given a context limit, an agent keeps the latest messages or tokens and discards older state. This is simple but ignores causal structure. An action and its observation form a unit. A source view may remain important after several later path-discovery commands. A mutation can remain relevant until a subsequent verification. A test result may be obsolete after another edit, while the task statement remains permanently relevant.

This paper isolates the memory-policy problem from the serving-engine problem. At model turn t , let the full canonical history be

$$H_t = [r_1, r_2, \dots, r_n],$$

where each r_i is a logical agent record. A memory policy computes

$$S_t = \pi(H_t, Q_t, B),$$

where Q_t is the active task/state and B is a context budget. The selected plan S_t is then serialized to an ordinary model request. No native K/V mechanism is required for the main experiments.

This separation is deliberate. Prior PRA work studies retrieval, native-memory materialization, cross-document composition, and engine-specific K/V reuse. Those systems questions should not determine which semantic records an agent needs. Here we first identify useful logical memory policies. A separate runtime paper can later ask whether different engines realize the same selected plan correctly and efficiently.

We begin with mini-swe-agent because its control loop is minimal: a linear message history is sent to the model, the model emits shell actions, the environment executes them, and observations are appended to the same history. This minimizes hidden harness state and makes memory intervention easy to audit.

Research questions. We ask:

1. What accuracy–saving frontier does simple matched recency establish, and how much additional removable-history headroom does an offline oracle reveal?
2. At matched model-visible tokens, does typed payload retirement that preserves provenance/control outperform whole-record semantic retirement and simple recency?
3. When bounded memory changes behavior, how much cost appears as direct reacquisition, trajectory inflation, submission failure, or loss of a resolving workspace state?
4. As one session grows from one to ten issues, which episode-retirement, selection, and materialization strategies preserve sequence-level quality, and do any retain useful cross-issue state more efficiently than a fresh session reset?

Contributions planned. We target five contributions:

1. An engine-independent typed record model that separates state, provenance, control, and semantic-evidence utility.
2. A matched-budget factorial over full history, recency, whole-record retirement, and payload-only retirement with provenance stubs.
3. A two-stage evaluation combining frozen-trajectory counterfactuals and oracle headroom with repeat-qualified autonomous SWE-bench runs.
4. Agent-specific efficiency endpoints that separate solution and submission reliability and charge direct reacquisition and trajectory inflation.
5. A multi-issue session frontier that reports every promoted strategy, configuration, and composition against both persistent-full and fresh-per-issue controls, with uncertainty clustered by ordered issue sequence.

2 Background and Motivation

2.1 Agent history differs from document retrieval

Document retrieval typically selects evidence that is externally stored and largely immutable. Agent memory contains generated and observed state whose meaning depends on the trajectory. A source observation may explain a later edit; an edit invalidates earlier verification; a rejected action may produce an error observation without becoming part of the accepted action history.

These dependencies motivate a record graph rather than an undifferentiated token stream.

2.2 Why mini-swe-agent

mini-swe-agent maintains a linear message list. At each step the model receives that list, returns one or more actions, the environment executes those actions, and formatted observations are appended. Because there is little additional memory management in the harness, the trajectory itself is the natural object for intervention.

We use mini-swe-agent as the first experimental harness, then test whether the highest-performing policies transfer to a richer open-source agent and to a record-native PRA agent.

mini-swe-agent exposes one generic Bash action interface. Consequently, tool choice is not a meaningful routing variable in this first study: the scientific variables are which action–observation history survives and how large tool observations are represented. We do not place a PRA gateway in the primary factorial. Gateway tool routing and result compaction are separate interventions that can be tested only after the logical memory policy is fixed.

3 Agent Record Model

We normalize trajectory messages into typed records:

$$\mathcal{R} = \{\text{TASK, ACTION, OBSERVATION, SOURCE, MUTATION, VERIFICATION, PROGRESS, ERROR, FINALIZATION}\}.$$

Each record contains a stable identifier, turn index, semantic type, causal group, source identity/spans where applicable, mutation target, and logical creation time.

3.1 State, provenance, control, and semantic utility

A record has several lifetimes rather than one scalar relevance value. For a record r at decision t , we use the conceptual decomposition

$$U(r, t) = U_{\text{state}}(r, t) + U_{\text{provenance}}(r, t) + U_{\text{control}}(r, t) + U_{\text{semantic}}(r, t).$$

U_{state} measures whether the payload authoritatively describes the current external world, such as a file at a particular version. $U_{\text{provenance}}$ records why the agent inspected or changed that resource. U_{control} carries progress, hypotheses, intended next steps, and finalization state. U_{semantic} captures facts useful for later reasoning even when they are no longer the current-state projection, such as an old traceback that exposes an architectural constraint.

The DAG can sometimes establish that a payload is operationally superseded, but this proves only a reduction in U_{state} . In particular,

$$\text{state superseded} \not\Rightarrow \text{record behaviorally useless.}$$

This distinction motivates partial retirement: retain action/provenance and a typed state transition while eliding replaceable bulk payload. It also explains why operational certificates and behavioral outcomes are separate measurements throughout the paper.

3.2 Causal bundles

Actions and observations are grouped:

$$\text{ACTION}_i \leftrightarrow \text{OBSERVATION}_i.$$

Mutations and later verification can form a higher-level dependency:

$$\text{MUTATION}_i \rightarrow \text{VERIFICATION}_j.$$

Policies may rank individual records, but required bundles are rounded up unless the experiment explicitly tests bundle-breaking.

4 Memory Selection Policies

4.1 Full history

FULL retains all canonical records and defines the reference trajectory.

4.2 Recency baselines

We test:

- **TOKEN-TAIL**: last B tokens;
- **TURN-TAIL**: last N complete turns;
- **RECORD-TAIL**: last M records;
- **CAUSAL-TAIL**: last N complete causal bundles.

4.3 Independent head, middle, and tail

Recency and early-run retention are separate variables. We therefore define a three-zone policy

$$S_t = I \cup H_h \cup \pi_{\text{middle}}(M_t, Q_t, B') \cup T_\tau,$$

where I is the immutable system prompt and all user-authored instructions, H_h contains the first h complete action–observation turns, T_τ contains the latest τ complete turns, and the selector operates only on the non-overlapping middle M_t . The head and tail controls are independent; $h = 0$ never removes I . Thus task definitions and later user prompts are not optimization candidates in this study. A tool observation transported through a user-role message remains selectable because its semantic type and provenance are **TOOL-OBSERVATION**, not **USER-INSTRUCTION**. Oversized user instructions may eventually be handled internally by an engine-side PRA implementation, but that is outside the logical-retirement intervention measured here. Our pilot grid uses

$$h \in \{0, 1, 2, 4\}, \quad \tau \in \{1, 2, 3, 5, 10\}.$$

Frozen replay eliminates dominated combinations before autonomous evaluation. This factorization tests whether early localization or plan formation remains useful after leaving a conventional recency window.

Head and tail turns remain causally complete. An oversized tool observation is the only permitted within-turn exception: its action, return code, resource identity, mutation/verification status, error headline, and causal identity remain present while its body may use a separately recorded detail policy. When immutable/head/tail state exceeds the requested budget, we report mandatory overflow and realized retention rather than silently breaking a turn.

4.4 Typed progress spine

The structured policy retains

$$S_{\text{spine}} = T \cup R_{\text{recent}} \cup S_{\text{source}} \cup M_{\text{mutation}} \cup V_{\text{verification}} \cup P_{\text{progress}},$$

where each component denotes the latest required records of that semantic role.

4.5 Persistent multi-issue sessions

A developer may solve several issues without opening a new conversation. We therefore separate the issue from the session: an *episode* is one issue and workspace lifetime, while a session is an ordered sequence of one or more episodes. Two modes consume the identical locked issue order: FRESH-PER-ISSUE assigns a new session identity to every episode, and PERSISTENT retains one identity across $n \in \{1, 2, 3, 4, 5\}$ episodes. Every transition declares the new active task, workspace scope, prior episode completion, and globally unique record, turn, and causal-group identities. Incompatible SWE-bench base commits remain separate workspaces; persistent conversation state must not be confused with a physically merged repository.

The first episode-aware policy keeps the active episode whole and retains each completed episode’s task statement, newest complete turn, latest mutation, and latest verification. An early aggressive diagnostic kept only the system prompt and active episode. That legacy arm is now superseded because it could remove an earlier user-authored task statement. Under the current invariant, “active only” can retire all completed assistant–tool interaction but still retains every user instruction. These episode policies operate before any within-episode token budget, so their saving can grow with the number of completed issues instead of being capped by a nominal 90% retention treatment.

Explicit episode boundaries are useful to an evaluator but are not a sound deployment assumption: developers need not announce that a new issue has started. We therefore also evaluate a *boundary-free global* policy over one continuous transcript. It receives no episode identifier, completion marker, or workspace-scope annotation. Every user-authored instruction is still immutable, while recent-turn, mutation, verification, and protocol floors are computed globally over typed assistant–tool causal groups. This arm isolates the realistic question: how much of the tool-interaction history can be retired when the selector is not told where one task ends and another begins?

4.6 Retrieval policies

We compare lexical, dense, and hybrid retrieval over historical records using a query formed from

$$\text{task} + \text{current progress} + \text{latest active observation}.$$

The hybrid policy combines a mandatory structured spine with retrieved records.

4.7 Resource/effect DAG and negative exclusion

Positive selection asks which records are likely useful. We additionally test the dual question: which records are operationally obsolete under a declared resource-state abstraction? We construct an acyclic graph over actions, observations, immutable resource versions, and precisely defined projections. Typed edges include RETURNS, READSFROM, WRITES, SUPERSEDED BY, and SUBSUMED BY. Effects carry provenance from declared tool semantics, runtime tracing, static heuristics, or UNKNOWN; unknown effects prohibit certified exclusion.

The two-stage policy is

$$H_t \xrightarrow{\text{certified exclusion}} H'_t \xrightarrow[\text{only if still over } B]{\pi_{\text{positive}}} S_t.$$

A certificate names the rule, witnesses, proof scope, resource-version fingerprints, cwd/environment identity, completeness conditions, and explicit assumptions. We separate three tiers: DAG-CERTIFIED, DAG-RECOVERABLE, and DAG-HEURISTIC. Only the first is removed by the conservative $B = 100\%$ exclusion arm; the others require the same counterfactual and autonomous qualification as positive retrieval.

The scope of “certified” is deliberately narrow. A later record can be proven to supersede an older current-state projection, but this does not prove that the older text contains no decision-relevant diagnostic fact. Moreover, deleting even byte-identical repeated text changes token positions and attention. Thus the claimed property is *certificate-backed operational obsolescence*, not risk-free behavioral equivalence. Behavioral preservation remains an empirical outcome measured by frozen replay and autonomous tasks.

The initial executable rule, TRACE-EXACT-OPERATION-RESULT, compares the executed command and complete observation rather than the assistant’s private reasoning prose. It fires only when cwd, environment identity, affected resource-version fingerprints, return code, and observation bytes agree under runtime-traced or declared-complete semantics. The older whole causal bundle is then operationally redundant. Different reasoning prose is intentionally outside the certificate, which is why frozen behavioral qualification remains required even for this strongest exclusion tier.

Initial rules fail closed: system/task/current state and unresolved errors are protected; arbitrary or untraced Bash is UNKNOWN; output must be complete and non-truncated; file reads require canonical paths and version/span identity; git queries require separate HEAD, index, and worktree identities; and action–observation bundles remain intact. A write invalidates an old record as an authority on current state but does not by itself authorize semantic deletion. Similarly, a later passing test does not automatically erase a prior failure’s diagnostic information.

4.7.1 Auditable negative-selection heuristic ladder

Certification coverage will initially be sparse, so we test four explicitly non-certified negative-selection families before invoking learned relevance. All retire complete action–observation groups, preserve independent head and tail floors, and emit a compact inactive tombstone containing the rule, resource, and witness identities. The tombstone is controller provenance and is not serialized into the model request; serializing it would be a distinct summarization treatment.

H1–CONSUMED SEARCH retires a discovery operation such as `find`, `fd`, `rg --files`, or filename-only `grep -l` after a returned resource is consumed by a concrete source read and the trajectory subsequently advances to a non-read operation. Source-bearing `grep -n` and ordinary `rg` matches count as reads, not discovery. The delay $K_f \in \{0, 1, 2, 4\}$ starts at that transition and measures whether the location evidence retains short-lived value after consumption. An unresolved search/read exploration chain is not retired.

H2–WRITE CONSOLIDATION is split because absence of a rewrite does not imply that a mutation was good. H2a requires an observed resource change and a later read or diff of the same post-write version. H2b requires a successful verification with an explicit dependency on the written resource; a workspace-wide passing check is a separately labelled relaxed arm. Only the raw write bundle is retired, while the later current-state or verification evidence remains. The bare “no later rewrite” rule is retained solely as an aggressive ablation with $K_w \in \{1, 2, 4\}$.

H3–READ SUPERSESSION retains the latest K_r observations, for $K_r \in \{1, 2, 3\}$, covering each resource–version–span key. A whole-file read covers a line span; a later disjoint `sed` region does not; `grep/tail` projections require the same query signature. Missing versions use a no-intervening-write epoch only in the heuristic tier and never become a certificate.

H4–BOUNDED WORKING SET retains the latest $K_x \in \{2, 3, 4, 6\}$ distinct read resources while pinning resources named by the task, mutated state, unresolved failures, the current tail, and declared dependencies. Cross-file software dependencies make H4 the riskiest family; it is reported as a capacity heuristic rather than safe exclusion.

The initial grids above target short trajectories. We separately predeclare a long-horizon grid with $K_f \in \{0, 2, 4, 8, 12, 16\}$, $K_w \in \{1, 2, 4, 8, 12\}$, $K_r \in \{1, 2, 3, 4, 6, 8\}$, and $K_x \in \{2, 3, 4, 6, 8, 12\}$. Its primary cohort contains only ordinary FULL executions that resolve officially, contain at least 25 model decisions, and exceed 100,000 cumulative exact-tokenizer input tokens. We report both the complete growing-prefix integral and suffixes beginning at decisions 10, 15, and 20. This tests a specific nonstationarity hypothesis: after workspace mutations, old observations may cease to be authoritative even though task intent, mutation rationale, and unresolved failures can remain useful. Age alone is therefore never treated as evidence of irrelevance.

The high-retention sweep uses targets 100%, 99%, 97.5%, 95%, and 90%. Every semantic arm is paired with a recency control using its exact per-decision materialized-token count; a fixed keep-last- K control is also reported to show how its realized retention changes with task length. This matched design asks whether resource/version/effect evidence removes stale state more efficiently than blind age, rather than merely comparing policies that remove different numbers of tokens.

The predeclared sequence isolates H1, H2a, H2b, H3, and H4 before combining H1+H3, H2+H3, H1+H2+H3, and finally all four. If the negative stage remains over budget, the original absolute budget is passed to the positive selector. This ordering identifies whether failure came from a particular retirement rule or from subsequent relevance selection.

4.7.2 Tool-semantics ownership and runtime handoff

mini-swe-agent exposes only one generic Bash tool, so the Paper 8.5 harness provides a Bash-specific semantics adapter that classifies common commands and, when available, augments them with runtime filesystem/version tracing. This special case belongs in the harness-facing adapter, not in a PRA inference engine. The first implementation certifies only simple allowlisted read or pure commands. Pipelines, lists, substitutions, redirections, heredocs, multiline scripts, timeouts, and truncated observations fail closed even when a coarse regex suggests a familiar command category.

The instrumented Docker environment leaves the model-visible observation text unchanged and attaches a versioned metadata envelope containing canonical `cwd`, environment identity, return and completeness status, pre/post resource fingerprints, and pre/post repository-state fingerprints. Before every action it also stores the base commit, separate binary index and worktree patches, and an archive of nonignored untracked files. Untracked file contents, not only their names, participate in the state digest. This is an exact, restorable repository state under the declared scope; ignored artifacts, container-global state, network state, and resources outside the repository remain explicitly out of scope. A restore verifier refuses incomplete receipts, dirty targets, digest mismatches, and

unsafe archive paths.

The portable interface is instead a tool-semantics provider. A developer or agent SDK can declare generic reads, writes, deletes, verification targets, resource versions, causal dependencies, output completeness, and confidence. It may also attach record directives such as protected, certified-excludable, or preferred-for-selection, with a certificate and witness IDs. Future tool categories implement the same effect vocabulary without teaching the engine about Bash, Excel, browsers, databases, or particular agent frameworks.

The implementation now has a shared mediation layer that can run inside a PRA runtime or, only when the engine cannot host it, in the external gateway. Both placements consume the same content-addressed `WireAgentMemoryPlan`; stale-history digests and conflicting second-hop mediation fail closed. Thus moving an unchanged plan between placements is a conformance test, not a reason to rerun the entire policy campaign.

The common state-authority planner consumes only declared operation classes, resource evidence (identity, version, and span), output completeness, and causal identities. It contains no mini-swe-agent or Bash parser. The mini-swe-specific compatibility code is confined to the evaluation adapter that translates its nonstandard assistant–user Bash exchange to this portable schema. Standard typed tool traffic, or an agent SDK that emits the schema directly, bypasses that adapter. An embedded runtime supplies the exact model tokenizer for receipt size gates; an external fallback must load the same tokenizer or consume an already frozen plan.

When the policy returns to Paper 4.5, the PRA engine validates and physically realizes selected logical identities; it does not infer application-specific semantics. Paper 8.5 freezes model-visible selected IDs and replacement bytes. Paper 4.5 then measures cache hits, K/V movement, re-encoding, latency, memory, and lifecycle on those same plans. Result compaction and tool disclosure are promoted independently from shadow, so later gateway capabilities do not silently invalidate unchanged history-policy evidence.

4.8 Oracle

A retrospective oracle estimates future usefulness through record/bundle ablation on successful trajectories. It is an upper bound and is never used as a deployment policy.

5 Materialization Policies

Record selection and record detail are separate variables.

For selected records, we compare:

- full record;
- matched source span;
- bounded head/tail;
- hierarchical child span;
- optional summary;
- compact provenance stub for an excluded record.

The first experiments use whole-record materialization so that selector effects remain identifiable. A targeted second factorial applies detail reduction only to oversized tool observations. Source

views retain matched symbol/file spans; test outputs retain the command, return code, failed-test identities, exception, and relevant stack frames; search results retain matched lines and locations; mutations retain their changed span or diff. This reuses gateway-style compaction semantics without introducing a gateway or an engine into the study.

Negative exclusion therefore has two separately measured realizations. The primary arm removes the retired payload from the model request while retaining its tombstone only in controller state. A second *model-visible stub* arm replaces it with compact metadata such as resource, mutation/verification status, and superseding-witness identity. The latter may preserve useful provenance, but it is a materialization/compression treatment rather than pure exclusion and its stub tokens count against the realized budget. This cleanly separates binary HOT→WARM retirement from HOT→STUB presentation.

6 Experimental Design

6.1 Recordizer reliability

Policies cannot be credited for semantic types that the recordizer identifies unreliably. We therefore separate tool-grounded labels (operation, resource, version, return code, mutation, and verification) from prose-inferred labels such as PROGRESS. A blinded worksheet has now sampled 150 of 164 records from three frozen task trajectories, stratified by task, automatic primary role, and early/middle/late trajectory position. It reports per-type precision and recall plus exact causal-group boundary agreement. Policy results expose both the declared/runtime-grounded view and the inferred semantic view; low-precision inferred roles cannot support a default profile. The worksheet and digest-bound source manifest are released under `recordizer_audit_v1`; the present draft makes no reliability claim until its human-label columns are complete.

6.2 Phase I: frozen-trajectory screening

We begin from successful full-history trajectories. At each historical model call we restore the corresponding workspace, apply a candidate memory policy, and query the same frozen model.

We measure:

- exact next-response agreement;
- exact action/command agreement;
- command equivalence under the frozen operation normalizer;
- target-resource agreement;
- semantic state-transition agreement (read, write, verification, or finalization with matching affected resources);
- final solution-state success for resumed rollouts;
- first-divergence turn;
- retained tokens and records.

Frozen screening permits broad policy and budget sweeps without paying for full autonomous reruns.

The first frozen cohort follows a gate rather than a Cartesian sweep. On the same instrumented successful trajectory, frozen model revision, tokenizer, temperature zero, generation parameters, and decision snapshots, we run:

1. FULL, establishing contemporaneous reproducibility;
2. DAG-EXCLUDE at a nominal 100% ceiling;
3. a recency/token-tail control matched to the DAG arm’s realized token count at each decision;
4. head–middle–tail policies at a 90% minimum-retention floor, rounded upward at whole causal-turn boundaries.

Failure of FULL stops interpretation of later arms because it means the frozen baseline itself was not reproduced. Autonomous qualification and oracle add-back tests begin only after this ordered next-action gate. The matched-tail control distinguishes the value of certified semantic exclusion from the value of merely removing the same number of old tokens.

The two budget contracts are intentionally different. The matched-tail arm must not exceed the DAG arm’s realized token count, so it rounds down and reports unused capacity. A treatment named “90%”, however, must retain at least $\lceil 0.9N \rceil$ logical tokens: it follows the policy ranking until the next indivisible causal bundle crosses that floor and reports the resulting whole-turn overshoot. A preliminary implementation treated 90% as a hard ceiling and retained only 71.9% on one decision; that partial run is quarantined as a budget-contract failure rather than interpreted as policy quality.

The repository state is part of the frozen decision point. Replaying turn t restores the exact pre-action repository state within the declared checkpoint scope, not merely the textual history, so a candidate command is evaluated against the same tracked, staged, worktree, and nonignored-untracked contents as the reference action.

6.3 Phase II: autonomous evaluation

Policies that survive frozen screening are run autonomously from a fresh task sandbox. Final success is measured by the official SWE-bench grader.

Autonomous evaluation is required because a divergent action may still recover later, while a seemingly small memory omission may cause many additional calls.

6.4 Session schedule factorial

For each promoted policy we run both FRESH-PER-ISSUE and PERSISTENT schedules over the same ordered issue sequences. Required controls are fresh FULL, persistent FULL, and persistent policy selection. Persistent FULL can itself degrade through cross-issue interference, so policy quality is compared both with its contemporaneous persistent control and with official fresh-session resolution. We report per-episode resolution, all-episodes resolution, calls, first cross-issue contamination, current-prompt saving, and cumulative input saving. The ordered issue sequence, not an individual request, is the uncertainty unit. Paper 4.5 reuses the same schedule identifiers for K/V and runtime measurements.

6.5 Benchmarks

We begin with the existing SWE-bench Verified Easy-14 task sequence, then expand to Easy-50 and a broader Verified sample.

Easy-14 is a mechanism/debug cohort. Publication-level claims require a larger task set.

6.6 Budgets

We evaluate relative budgets

100, 90, 75, 50, 25%

and absolute budgets

2K, 4K, 8K, 16K, 32K.

Every request records both requested and realized retention.

7 Metrics

7.1 Task outcome

We retain official SWE-bench resolution as the primary benchmark endpoint, but factor it into two separately reported agent outcomes:

$$Y_{\text{official}} = Y_{\text{solve}} \wedge Y_{\text{submit}}.$$

Y_{solve} asks whether the final workspace contains a resolving change; Y_{submit} asks whether the agent correctly emits that change through the required terminal protocol. An auxiliary workspace grade can identify $Y_{\text{solve}} = 1, Y_{\text{submit}} = 0$, but never replaces the primary official result. If the final workspace cannot be reconstructed under the fail-closed provenance rules, Y_{solve} is unknown rather than zero. Additional endpoints are calls to completion, shell actions, and final patch/test status.

7.2 Memory efficiency

We record full-history tokens, selected tokens, retention fraction, selected record count, selected records by semantic role, and the largest retained record.

For DAG policies we additionally report certified, recoverable, heuristic, and unknown-effect tokens separately; certificate coverage and abstention; records removed per rule; and certificate violations under frozen add-back tests. A certificate can be operationally sound while the LLM still changes behavior, so operational certificate validity and next-action survival are distinct metrics.

We distinguish two savings quantities. For an autonomous candidate trajectory c , gross history saving is

$$G_c = 1 - \frac{\sum_t M_{c,t}}{\sum_t F_{c,t}},$$

where $M_{c,t}$ is materialized input and $F_{c,t}$ is the full-history counterfactual at the same candidate call. Gross saving isolates the memory policy but does not charge for a longer trajectory. Against a declared paired FULL execution f , paired trajectory message-input saving is

$$N_{c,f} = 1 - \frac{\sum_t M_{c,t}}{\sum_t M_{f,t}}.$$

N includes extra or avoided calls, but covers only model-visible message content. It excludes chat-template tokens, completions, tool execution, and runtime; we therefore do not call it end-to-end saving. A failed run that stops early can have a favorable N . For quality–efficiency frontiers we use a failure-aware coordinate that charges an unsuccessful candidate at least the paired FULL message-input workload, so early failure cannot be reported as an efficiency gain.

Direct recovery is also charged separately. Let R_c be the token size of tool observations produced by actions that reacquire previously excluded resources. We report the diagnostic lower bound

$$G_c^{\text{reacq}} = \frac{\sum_t (F_{c,t} - M_{c,t}) - R_c}{\sum_t F_{c,t}}.$$

This is not added to $N_{c,f}$: the paired trajectory quantity already includes tokens replayed by a longer candidate trajectory. Instead, we expose recovery as two complementary components,

$$C_{\text{recovery}} = C_{\text{reacquire}} + C_{\text{trajectory}},$$

where the first is observed reread/search/test payload and the second is the paired call and total-input-token inflation. Their overlap is reported rather than summed into a potentially double-counted scalar.

7.3 Quality-saving and interaction curves

The primary frontier plots official task resolution against G and N , with G^{reacq} as a recovery-adjusted diagnostic. Trajectory agreement is a diagnostic proxy, not accuracy: a different action may still produce a resolving patch. We therefore report separate curves for (i) official resolution versus saving, (ii) tool-call delta versus saving over all paired runs, (iii) tool-call delta restricted to pairs where both the candidate and its declared FULL control resolve, and (iv) first tool-action divergence versus saving for every task. The divergence abscissa uses only saving accumulated on requests strictly before the first different action; tokens saved after divergence cannot explain that divergence and are excluded from this plot. Auxiliary terminal-workspace grading diagnoses submission failures but never replaces official resolution.

Because the tested endpoint is not autonomously deterministic even at temperature zero and a fixed seed, final quality curves use repeated paired FULL controls. Resolution is first averaged within task identity and then across tasks; confidence intervals bootstrap task identities, never individual executions. We report endpoint and exact-action agreement between the FULL repeats as an instability control. A conditional preservation analysis additionally evaluates candidates only on declared pairs for which FULL resolves. This avoids treating a baseline failure as a memory policy failure while retaining the unconditional resolution curve as the primary population result.

The current autonomous curve is deliberately underpowered and is reported as a pilot, not a profile result. In the latest same-host three-task cohort, repeated FULL controls resolve 5/6 and matched recency resolves 4/6; the latter misses the predeclared 80% task-resolution gate. Earlier H3 runs that failed early remain diagnostic points, but failure-aware saving assigns them no efficiency gain. The manuscript therefore keeps raw gross saving, paired trajectory saving, and qualification status as separate fields.

7.4 Recovery behavior

A memory policy may preserve final task success by forcing the agent to reacquire forgotten state. We therefore count repeated source reads, overlapping source rereads, repeated searches for the same fact, repeated tests without relevant intervening mutation, and repeated failed edits. We call this *reacquisition overhead*.

For every negative arm we also report exclusions and tokens by rule, the resource and witness provenance stored in inactive tombstones, and the fraction of exclusions whose information is later reacquired. Frozen replay includes a narrow immediate false-exclusion proxy: the selected-history generation reads a hidden resource when contemporaneous FULL does not. This proxy is not a

causal error rate—the primary evidence remains autonomous reacquisition, calls-to-solution, and official task success.

7.5 Dependency horizon

For a useful historical record r_i consumed at turn t , define

$$d(r_i, t) = t - \text{created}(r_i).$$

We report dependency-age distributions by record type. This estimates how long coding-agent memory must remain accessible.

8 Adaptive Policy Campaign

Every reported strategy has a stable identifier, a predeclared rationale, configuration coordinates, primary matched comparison, predicted failure mode, and implementation status in `multi_issue_strategy_registry_v1.json`. We do not execute a blind Cartesian product. Simple families are first isolated at two and three issues; a configuration that is low-saving, quality-losing, or call-inflating is stopped. A mixture is admissible only when each parent has an interpretable effect and at least one parent is nondominated. The combination then changes one coordinate at a time and is compared with each parent plus a matched-token tail. Only nondominated configurations advance to four and five issues and to held-out sequences. This preserves an argument for every tested cell without turning exploratory search into confirmatory evidence.

Table 1: Adaptive campaign stages. Frozen agreement prunes experiments but never substitutes for autonomous quality.

Stage	Arms	Advancement rule
Mechanism	Every simple rule	Predicate, serialization, accounting, and fail-closed tests must pass.
Frozen discovery	Simple rules and matched controls	Stop low-saving arms with rapid action-proxy loss; retain mechanism controls without promoting them.
Bounded autonomous	Nondominated or information-preserving candidates	Run one to three tasks with repeated FULL; require official success before task expansion.
Combination search	Only promoted parents	Hill-climb one parameter at a time; do not combine stopped or dominated arms.
Confirmation	Frozen policy set on held-out tasks	Report task-level quality, gross/net saving, calls, recovery, and uncertainty.

8.1 From a frontier to deployable profiles

The curve is the product interface, not merely a paper visualization. A developer needs official resolution, calls-to-solution, divergence, and rediscovery; an operator needs fallback frequency, realized retention, and an auditable per-session policy decision; an infrastructure buyer needs failure-aware saving, cost per resolved issue, concurrency capacity, and a quality confidence interval. We therefore predeclare four logical destinations without assigning a winning policy in advance: AGENT-FULL is the no-selection correctness control; AGENT-QUALITY, AGENT-BALANCED, and

AGENT-ECONOMY choose the highest-saving nondominated frozen point subject to lower sequence-clustered quality-delta bounds of -2 , -5 , and -10 percentage points, respectively. The latter also requires at least 20% failure-aware saving. Balanced and Economy must improve cost per resolved issue; Quality and Balanced must not materially inflate calls among joint successes.

These names remain candidates until held-out autonomous confirmation. Frozen next-action agreement alone cannot promote a profile, and insufficient data cannot be converted into confidence by pooling requests, repeated executions, or different sequence strata. The complete logical-to-runtime contract is versioned in `paper4_5_profile_promotion_contract_v1.json`. Paper 4.5 may claim a runtime-qualified profile only after realizing the identical frozen logical plan while independently passing 100% parity, sparse-K/V identity, and lifecycle gates.

The adaptive rule is versioned before each discovery stage. Current whole-group H1/H2a combinations are stopped because they lose the action proxy rapidly for small savings; H4 remains an aggressive mechanism bound. Matched token tail and positional materializers remain controls rather than parents. Structured evidence receives a bounded autonomous probe because it preserves all causal groups, but it does not expand unless task quality passes. This pilot-informed pruning is exploratory. Final profile claims use a subsequently frozen policy set and held-out tasks, preventing adaptive search from being mistaken for confirmatory evidence.

8.2 Pilot structural screen

Before issuing counterfactual model calls, we ran the recordizer and selector over three successful mini-swe-agent reference trajectories containing 69 historical model decisions. The full grid contains five budget fractions, four head floors, five tail floors, five initial middle policies, and two DAG-first compositions, for 48,300 selection decisions and 2,100 aggregated cells. This screen uses a whitespace counter and is therefore evidence only about structural feasibility, not model token counts or policy quality.

Table 2 shows one illustrative slice with one first turn and two latest turns retained. Whole causal bundles leave some budget unused. Conversely, immutable and mandatory state can exceed the ceiling. At a 90% request the basic head-tail policy overflows on 16 of 69 decisions, while the role-aware progress spine overflows on 22 of 69. These are expected round-up outcomes, concentrated when the trajectory is short; they demonstrate why requested percentage, realized retention, and mandatory overflow must be reported separately.

The legacy trajectories lack cwd, environment, output-completeness, and resource-version fingerprints. The certificate-backed DAG arm therefore correctly abstains and retains 100% at $B = 100\%$. Static Bash analysis exposes 458 superseded-current-state and 75 write-invalidated-state candidate occurrences across the 69 growing prefixes, but no trace-exact duplicate and no certified exclusion. These are repeated candidate occurrences, not unique records, and are not removed. This is an instrumentation requirement, not a negative quality result.

Table 2: Aggregate realized retention in the structural-only pilot for $h = 1, \tau = 2$. These values do not measure next-action or task quality.

Policy	90%	75%	50%	25%
Middle none	55.5%	55.5%	55.5%	55.5%
Progress spine	66.5%	66.5%	66.5%	66.5%
Middle recency	85.9%	76.7%	59.6%	55.5%
Middle lexical	89.5%	77.1%	59.6%	55.5%
Spine + lexical	91.6%	80.0%	67.4%	66.5%

8.3 Persistent-session structural opportunity

We composed three already successful real trajectories into an ordered multi-issue session and counted tokens with the frozen Qwen3-Coder tokenizer. This is a structural opportunity screen only: no replay or autonomous quality claim follows from it. Table 3 contrasts full persistent history, a conservative completed-episode spine, and retirement of all completed episodes. The latter is intentionally an aggressive diagnostic.

Table 3: Tokenizer-exact structural opportunity as completed issues accumulate. Savings are relative to persistent FULL; task quality and K/V reuse remain unmeasured.

Policy / issues	Final selected/full	Final save	Cumulative selected/full	Cumulative save
Completed spine / 2	14,681/21,989	33.23%	380,891/534,359	28.72%
Completed spine / 3	15,381/33,431	53.99%	632,632/1,147,100	44.85%
Active only / 2	12,237/21,989	44.35%	329,567/534,359	38.32%
Active only / 3	11,206/33,431	66.48%	497,808/1,147,100	56.60%

The displayed active-only rows are retained as a historical structural upper bound but are not eligible policy evidence: their legacy configuration allowed completed task statements to be removed. Every subsequent experiment pins all user-authored instructions. Tool observations remain selectable based on typed provenance even when the harness transports them with the user role.

Three deliberately fresh FULL sessions consume 496,540 cumulative message-content tokens. The active-only persistent arm consumes 497,808, only 1,268 tokens (0.26%) more, while preserving a single logical session identity. Thus it provides essentially no logical-token saving over deliberate fresh resets; its value is only automatic session hygiene for a developer who does not reset the conversation. This supports the proposed workload axis but not yet the stronger claim that persistence improves either agent quality or an engine: native K/V reuse, original-position sparse correctness, prefix-cache comparison, and agent-level resolution are deferred to Paper 4.5 after Paper 8.5 quality gates pass. Auditable schedules, composed traces, and decision rows are released under `multi_issue_session.v1`.

The decisive experiment therefore uses two additional strata. Related issues share a repository and can benefit from prior localization or architectural knowledge; dependent issues share a workspace and may consume artifacts or decisions produced earlier. On independent cross-repository issues, FRESH-PER-ISSUE is expected to dominate logically and serves as a negative control. On related and dependent sequences, a selective persistent policy can be credited over fresh reset only if reduced rediscovery or improved quality pays for every retained prior-episode token.

A one-decision frozen smoke further demonstrates why structural opportunity is not a policy frontier. At two issues, completed-spine and active-only retention materialize 35.02% and 13.29% of persistent history, but neither reproduces the contemporaneous FULL command. At three issues, the same policies materialize 28.45% and 11.90% and both reproduce that command. In both cases the contemporaneous FULL call itself differs from the command stored in the historical successful trajectory. The opposite outcomes and unstable historical control prohibit a policy ranking; the artifact is retained only as a mechanism and instability audit. Repeated autonomous official resolution is the next quality gate.

8.4 Repeat-confirmed two-issue persistent-session point

An infrastructure-clean autonomous persistent-session experiment now provides a repeat-confirmed point beyond the structural screen. The ordered cross-repository sequence is `django-15277` followed

by `pytest-7982`, with a clean workspace for each issue but one persistent conversation identity. Both persistent FULL repeats resolve both issues. The R4/M2/V2 completed-episode spine keeps the completed task statement, four recent completed turns, two mutation turns, and two verification turns. Its first issue is the paired FULL episode by exact path and SHA-256 identity; selection begins only for the second issue.

Table 4: Infrastructure-clean autonomous N=2 repeat evidence. Candidate gross saving uses the candidate’s own realized trajectory. Paired failure-aware saving charges selected tokens against the declared persistent FULL trajectory and therefore also reflects avoided or extra calls.

Arm	Resolved	Calls	Full tokens	Selected tokens	Gross save	Paired save
Persistent FULL, repeat 1	2/2	42	330,537	330,537	0.00%	–
Persistent FULL, repeat 2	2/2	39	292,686	292,686	0.00%	–
R4/M2/V2, paired to repeat 1	2/2	35	253,582	209,878	17.23%	36.50%
R4/M2/V2, paired to repeat 2	2/2	35	253,582	209,878	17.23%	28.29%

All admitted traces have contiguous request indices, zero upstream failures, the frozen model and tokenizer revisions, and official SWE-bench grading. On the selected second issue, R4/M2/V2 diverges at the first action but resolves in eight calls in both repeats, versus 15 and 12 calls for paired FULL. Thus exact trajectory agreement is not the utility endpoint. Across the two paired sequences, the ratio-of-sums message-input saving is 32.65%, calls fall from 81 to 70, and no FULL success is lost. The macro mean of the two paired savings is 32.40%. This passes the predeclared 30–50% gate, but the uncertainty unit remains one ordered sequence family: the repeats measure execution stability, not population accuracy.

The first N=3 expansion does not qualify. Its persistent-FULL controls resolve only 2/3 issues in both repeats because `sphinx-8721` fails, so this sequence cannot establish preservation of three successes. R4/M2/V2 likewise resolves 2/3, but its paired savings vary from –37.40% to 47.70% as calls vary from 67 to 40. A single exploratory R2/M2/V2 execution reaches 45.31% paired saving and 42 calls, but also resolves only 2/3. These results expose trajectory-cost variance and motivate replacing the third issue with a repeat-qualified FULL success before further profile promotion. The compact confirmation ledger, task-aware pairing identities, artifact hashes, reductions, and curves are released under `autonomous_persistent_spine_confirmation_v1`; the earlier `autonomous_multi_issue_frontier_v1` remains the discovery record.

8.5 Replacement N=3 reliability and the R4 rejection

We replaced the non-repeatable Sphinx issue with `scikit-learn-13135` and separated task capability from submission reliability. One preserved FULL repeat produced correct saved workspace patches for Django, Pytest, and Scikit-learn, but emitted non-diff terminal payloads. Its primary score is therefore 0/3; counterfactual official regrading of the saved workspace patches is 3/3 and is reported only as auxiliary workspace capability. We do not retry this observation away or substitute the auxiliary score for unconditional resolution.

After correcting the grading path, three independent infrastructure-clean persistent-FULL repeats resolve all three issues: 9/9 issue resolutions and 3/3 all-solved ordered sequences. Each uses 53 calls, with 522,479, 550,355, and 553,263 cumulative logical input tokens. The artifacts are distinct by path, timestamp, and SHA-256 identity. Health and transport failures are retained in immutable retry directories but excluded from this denominator.

Table 5: Replacement N=3 R4/M2/V2 gate. R4 shares issue 1 by exact artifact identity and starts selection at issue 2. Paired raw saving charges R4 selected tokens against its corresponding persistent-FULL control; failure-aware saving is zero for both failed pairs.

Arm	Resolved	Calls	Full tokens	Selected tokens	Gross save	Paired raw
FULL, repeat 1	3/3	53	522,479	522,479	0.00%	–
R4/M2/V2, repeat 1	2/3	49	539,956	377,476	30.09%	27.75%
FULL, repeat 2	3/3	53	550,355	550,355	0.00%	–
R4/M2/V2, repeat 2	2/3	45	439,877	311,333	29.22%	43.43%
R4 aggregate	4/6	94	979,833	688,809	29.70%	35.80%
Paired FULL aggregate	6/6	106	1,072,834	1,072,834	0.00%	–

R4 reaches the target 30–50% interval only under the paired raw coordinate and reduces calls from 106 to 94, but it loses the Scikit-learn success in both repeats. Unconditional R4 resolution and conditional preservation on paired FULL successes are therefore both 4/6. The predeclared failure-aware saving is 0%, and the zero-loss quality gate rejects the arm regardless of its apparent efficiency. In one failure, the agent identifies the correct `np.sort` edit, executes a regex-based `sed` command that does not match, verifies the unchanged line, and submits an empty diff. In the other, it submits the right semantic operation at invalid indentation. These are task-level execution failures, not grader failures.

The repetitions also expose an important control result. The first three R4 Pytest requests have identical selected-message hashes and token counts across repeats, but different response hashes despite temperature zero. Likewise, the first R4 Scikit request is byte-identical across repeats and already produces different responses. Temperature zero is therefore not a bitwise replay guarantee for this backend. Policy claims require repeated official outcomes; exact logical-input identity is an audit control, not a replacement for replication. More conservative R6 and R8 arms are evaluated only as adaptive repairs after this predeclared R4 rejection. R6/M2/V2 restores 3/3 official resolution in its first execution, but Scikit-learn expands to 27 calls and the ordered sequence to 63 calls versus 53 for paired FULL. R6 materializes 538,318 of 807,886 tokens on its own inflated trajectory, a 33.37% candidate-gross saving, yet paired FULL consumes only 522,479 tokens. Its paired raw and failure-aware saving are both -3.03% , so it fails the saving and no-call-increase gates. This is precisely why a retention-only curve is not a utility frontier: the policy can remove a large fraction of each prompt while causing enough extra decisions to increase total consumption. An independent R6 execution reverses the apparent tradeoff: it uses 46 calls and materializes 326,252 of 456,267 candidate tokens, for 28.50% candidate-gross and 37.56% paired-raw saving, but loses the paired Scikit-learn success. Its failure-aware saving is therefore zero. Across the two R6 executions, official resolution is 5/6 versus 6/6 for paired FULL, and calls total 109 versus 106. R6 is not a reliable repair.

R8/M2/V2 completes 3/3 issues. It uses 59 calls, materializes 454,066 of 644,542 candidate tokens (29.55% candidate-gross saving), and saves 13.09% against its 522,479-token paired FULL. The extra six calls and the sub-30% paired saving reject it under both remaining gates. Table 6 keeps candidate-local compression separate from paired utility.

Table 6: Adaptive N=3 repairs. Failure-aware saving is zero for an execution that loses a paired FULL success; each row remains visible.

Arm	Resolved	Calls	Selected	Gross save	Paired raw	Failure-aware
R6/M2/V2, execution 1	3/3	63	538,318	33.37%	−3.03%	−3.03%
R6/M2/V2, execution 2	2/3	46	326,252	28.50%	37.56%	0.00%
R8/M2/V2, execution 1	3/3	59	454,066	29.55%	13.09%	13.09%
R4/M2/V2/P1, execution 1	3/3	47	332,269	30.02%	36.41%	36.41%
R4/M2/V2/P1, execution 2	3/3	44	299,078	28.23%	45.82%	45.82%

Figure 1 makes the gate explicit in the primary failure-aware coordinate. R4 is charged zero saving after losing paired successes; the two R6 outcomes average -1.52% failure-aware saving after ordered-sequence-family clustering; R8 remains well below the target band. P1 is the first arm to enter the target band while preserving every paired success: its two executions average 41.11% failure-aware saving and 7.5 fewer calls. These points are descriptive measurements of one ordered three-issue family, not a population interval. Repeated executions measure reliability and are not counted as independent task identities.

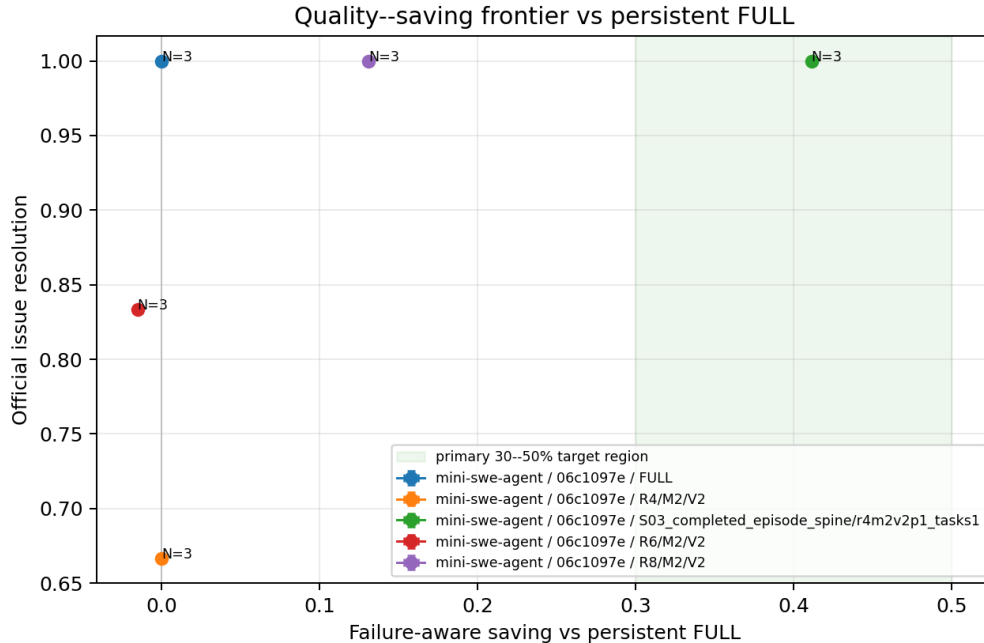


Figure 1: Failure-aware quality–saving frontier for the replacement N=3 cohort. The shaded band is the predeclared 30–50% target; P1 enters it while preserving all paired successes in both executions. Error bars are absent because this cohort contains only one ordered sequence family.

8.6 Frozen Scikit first-divergence oracle

The autonomous outcomes alone cannot separate selection from backend trajectory variance. We therefore freeze the R4 repeat-1 completed-episode prefix and reconstruct the exact first Scikit-learn selected-message digest. On this identical prefix, full materialization produces a valid Bash action in 5/5 requests, although two response and command hashes occur. R4/M2/V2 produces the same format-invalid response in 5/5 requests. Temperature-zero backend variance is real, but it does not explain this stable arm difference.

The R4 prompt already contains every available prior verification and finalization record and the two latest mutation turns per completed issue. We then restore each of its 25 retired causal groups separately. The restorations range from 82 to 2,513 tokens; 24/25 recover one valid action, and every valid action is a semantically equivalent source-file search represented by three exact command hashes. Only a 139-token mutation turn remains invalid. This is strong evidence for a selection-induced prompt-conditioning failure, but not for one missing Scikit-relevant fact: many unrelated old groups repair the protocol. Oracle trials remain diagnostic and do not enter policy-quality curves.

The deployable counterfactual is consequently content-independent: retain one clean completed action–observation exemplar per prior issue in addition to the recent, mutation, verification, and finalization floors. We denote this R4/M2/V2/P1. P1 selects the latest complete turn containing both a valid assistant action and tool observation while excluding error, mutation, verification, and finalization roles. It uses no future action and is not the oracle choice. The digest-bound oracle and guardrails are released under `oracle_scikit_r4_r01_first_request_v1`.

Two independent autonomous P1 executions pass the predeclared discovery gate. Both paired FULL controls resolve 3/3 in 53 calls. P1 resolves 3/3 in 47 and 44 calls, materializing 332,269 and 299,078 input tokens. Relative to contemporaneous FULL, the savings are 36.41% and 45.82%, with six and nine fewer calls. The descriptive means are 41.11% saving and 7.5 fewer calls. Candidate-trajectory gross savings are 30.02% and 28.23%; the difference from paired savings reflects avoided decisions rather than hidden token accounting. This result qualifies P1 for frozen transfer to the Paper 4.5 native-K/V evaluation *when equivalent episode metadata is available*. It does not yet establish generality across boundary-free sessions, ordered sequences, agents, or model families.

8.7 Boundary-free global retention

The preceding policy is allowed to know which completed turns belong to each issue. We next remove that signal. The model and selector receive one continuous transcript with no episode identifier, completion marker, or workspace-scope annotation. The immutable floor contains the system prompt and every user-authored instruction; mini-swe-agent tool observations remain selectable even though its compatibility protocol transports them using the user message role. R4/M2/V2/P1 is then applied globally to assistant–tool causal groups. Thus this treatment measures retirement of interaction history, not accidental deletion of a task statement.

Table 7: First autonomous boundary-free N=3 result. “Candidate gross” uses the candidate’s own unpruned trajectory counterfactual. Paired saving charges materialized candidate tokens against contemporaneous persistent FULL; failure-aware saving is zero after any lost paired success.

Policy	Resolved	Calls	Full tokens	Sent tokens	Gross	Paired / aware
Persistent FULL	3/3	55	599,397	599,397	0.00%	0.00% / 0.00%
Global R4/M2/V2/P1	2/3	66	748,306	293,091	60.83%	51.10% / 0.00%

The aggregate compression is therefore not a viable saving. Django falls from a 27-call success to an unresolved 40-call cap; Pytest improves from 14 to 9 calls and remains resolved; Scikit-learn remains resolved but rises from 14 to 17 calls. Django’s first action divergence occurs at request six, the first request after a causal group is retired, while still in the first issue. No prior-issue boundary exists at that point. This localizes the failure to an overly short global tool-history floor rather than missing task-boundary metadata or a removed user prompt. The predeclared adaptive response is to retain more interaction history (global R8) and rerun the complete paired cohort; R4 is rejected regardless of its 51.10% raw paired saving.

This failure exposes a conflation in a flat global floor. It uses the same parameter both to protect the active issue and to retire completed work. We therefore introduce a two-level *instruction-epoch* policy. A genuine provenance-typed user instruction begins an observable epoch; a tool result transported with the user role does not. The policy keeps the system prompt, every user instruction, and the entire current epoch. Only assistant–tool causal groups before the latest instruction are eligible for retirement. There is no evaluator task ID, episode marker, completion flag, or workspace label in the selector input. For independent repositories, the E0 treatment retires all such older interaction detail. For related or dependent work, that retirement is only an opportunity oracle until resource-version and dependency edges certify disconnection; otherwise the deployable policy must fail closed.

Before spending another autonomous cohort, we replayed the clean 3/3 persistent-FULL trajectories through E0 with the exact locked Qwen tokenizer. Table 8 counts every growing request. It is fixed-trajectory logical opportunity, not evidence that a selected model run still solves the tasks.

Table 8: Instruction-epoch opportunity on the clean N=3 persistent-FULL sequence. “Interaction saving” excludes the immutable system/user instruction floor from both numerator and denominator.

Issues	Requests	Full tokens	E0 tokens	Whole saving	Interaction saving
1	27	153,654	153,654	0.00%	0.00%
2	41	328,401	232,193	29.30%	42.72%
3	55	599,397	356,091	40.59%	59.56%

At N=3, immutable instructions account for 31.85% of cumulative content tokens. This distinction resolves the apparent “ten tasks imply 90% saving” argument. If equal-size independent tasks retain only the active assistant/tool epoch, the ideal saving *within interaction history* is $1 - 1/N$ for the *final request*, reaching 90% at N=10. It is not the cumulative-session saving: in the one-equal-request-per-epoch abstraction, that quantity is $1 - 2/(N + 1)$, or 81.8% at N=10. Real sessions weight each epoch by its number and size of growing requests. Whole-request saving is smaller again because system and user instructions remain pinned, and their share can grow with N . We therefore report final-request opportunity, cumulative own-trajectory saving, and paired end-to-end saving separately. The active autonomous E0 run is the behavioral test: any first-issue difference remains backend variance because E0 is an exact logical no-op until the second genuine instruction. The exact evidence is released under `instruction_epoch_oracle_n3_clean_full_v1`.

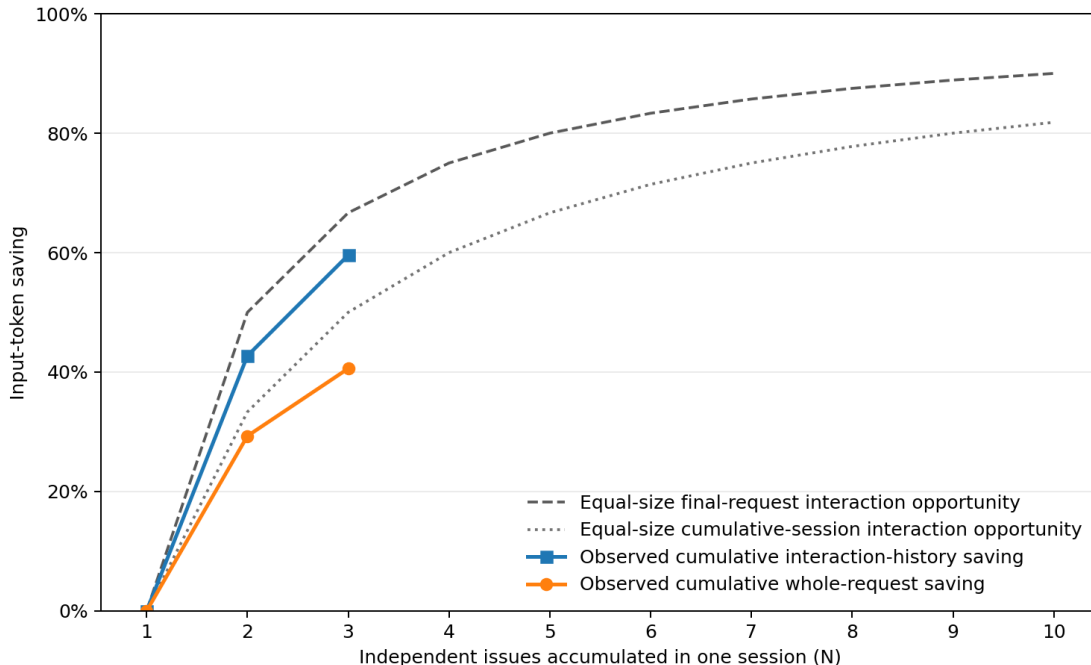


Figure 2: Empirical cumulative fixed-trajectory opportunity at $N=1-3$ and analytic equal-size interaction-history scaling to $N=10$. The dashed curve is the final-request limit $1 - 1/N$; the dotted curve is the cumulative-session abstraction $1 - 2/(N + 1)$. Neither is a whole-request quality forecast.

The first autonomous E0 execution lands inside the desired saving region but does not qualify because its contemporaneous control is not a clean accuracy baseline. Persistent FULL resolves only Scikit-learn (1/3); Django and Pytest end in grader/submission errors. E0 preserves the Scikit-learn success, resolves Pytest, and reproduces all 17 Django actions exactly because the first issue is a logical no-op. Table 9 reports both denominators.

Table 9: First autonomous instruction-epoch E0 execution. Paired saving uses the contemporaneous persistent-FULL denominator; candidate gross uses E0’s own unpruned trajectory. Both rows are evidence-inadmissible for a quality claim because the control contains grader errors.

Policy	Resolved	Calls	Full tokens	Sent tokens	Gross	Paired
Persistent FULL	1/3	47	623,656	623,656	0.00%	0.00%
Instruction epoch E0	2/3	44	567,826	320,375	43.58%	48.63%

Thus E0 loses zero contemporaneous FULL successes, reduces calls by three, and meets the 30–50% paired-saving interval. It nevertheless reaches only 66.7% unconditional resolution, below the predeclared 80% gate, and the sequence-level evidence is inadmissible. We preserve rather than retry away that baseline instability and run a fresh paired replication. A positive claim requires a repeat-qualified FULL control, not merely conditional preservation of one success. The complete first execution is released under `autonomous_persistent_instruction_epoch_n3.e0.v1`.

A fresh replication supplies the missing clean control and passes every predeclared discovery gate. Persistent FULL resolves 3/3 in 66 calls. E0 shares issue 1 exactly—a valid optimization because no earlier instruction epoch exists—then independently executes issues 2 and 3. It resolves 3/3 in 64 calls, sends 506,776 tokens, and has an 839,601-token own- trajectory counterfactual. Its

paired saving is 34.83% and its own-trajectory gross saving is 39.64%. Pytest falls from 14 to 10 calls; Scikit-learn rises from 15 to 17, leaving two fewer calls in aggregate.

Table 10: Clean autonomous N=3 instruction-epoch replication. Both rows are admissible; E0 loses no official success and remains inside the frozen 30–50% paired-saving interval.

Policy	Resolved	Calls	Full tokens	Sent tokens	Gross	Paired
Persistent FULL	3/3	66	777,645	777,645	0.00%	0.00%
Instruction epoch E0	3/3	64	839,601	506,776	39.64%	34.83%

This is the first valid boundary-free point in the requested quality-saving region. It qualifies E0 for repeat and sequence expansion, not for a universal default: retirement assumes independent instruction epochs, the uncertainty unit is one ordered sequence, and related/dependent work still requires a resource-DAG disconnection certificate or fail-closed retention. The evidence is released under `autonomous_persistent_instruction_epoch_n3_e0_replication_v1`.

Before expanding to N=6, we screened six additional short Easy-14 identities under ordinary FULL and then repeated every first-pass success. All six resolve in the first screen, but only three resolve again: SymPy-16886, Scikit-12585, and Django-16145. Scikit-14496 and Django-14089 are genuine second-run failures, while Django-15368 ends in an inadmissible `patch_apply_failed` grader/submission error. Calls also vary despite temperature zero—for example, Scikit-12585 increases from 14 to 19 calls. Thus “baseline-success task” is not a deterministic task label, and a policy comparison must expose unconditional resolution and repeat reliability rather than conditioning silently on one convenient FULL trajectory.

Table 11: Current-endpoint N=1 qualification screen. The second execution is preserved even when it fails; only identities with two official successes advance.

Identity	Calls A	A	Calls B	B	Advances?
SymPy-16886	9	pass	10	pass	yes
Scikit-12585	14	pass	19	pass	yes
Scikit-14496	13	pass	22	fail	no
Django-15368	16	pass	21	error	no
Django-14089	22	pass	40	fail	no
Django-16145	26	pass	26	pass	yes

The N=6 sequence combines those three new repeat-qualified identities with Django-15277, Pytest-7982, and Scikit-13135, each of which already has multiple admissible FULL successes in the released persistent-session evidence. The selector still receives no task ID, episode ID, completion flag, or workspace scope: it observes only ordinary message roles and record provenance. Screen evidence is released separately under `instruction_epoch_n1_candidate_screen_v1` and `instruction_epoch_n1_candidate_repeat_v1`.

The autonomous N=6 execution exposes the distinction between local retention and end-to-end efficiency. E0 resolves all six issues, including one issue that the contemporaneous FULL control misses, and its own growing trajectory sends 55.34% fewer tokens than that same trajectory without retirement. However, E0 takes 114 calls versus FULL’s 60. Its 1,171,988 sent tokens exceed the control’s entire 814,617-token workload, so paired failure-aware saving is -43.87% , not $+55.34\%$. The run loses none of the five FULL successes, but it fails the no-call-increase and 30–50% net-saving gates.

Table 12: Autonomous N=6 instruction-epoch result. “Gross” uses each row’s own trajectory counterfactual; “paired” compares sent tokens with the contemporaneous persistent-FULL workload and therefore includes the cost of extra calls.

Policy	Resolved	Calls	Full tokens	Sent tokens	Gross	Paired
Persistent FULL	5/6	60	814,617	814,617	0.00%	0.00%
Instruction epoch E0	6/6	114	2,624,244	1,171,988	55.34%	−43.87%

This is a useful negative result rather than a failed mechanism: retirement fires exactly as specified and preserves official resolution, but the model uses the shorter context less efficiently. We therefore predeclare E1 and E2, which keep the immediately preceding one or two genuine-user instruction epochs whole. These policies consume no task identifier; they use only the ordinary user-instruction boundaries already present in the transcript. The N=6 E0 evidence is released under `autonomous_persistent_instruction_epoch_n6_e0.v1`.

The completed E1/E2 ladder rejects both repairs (Table 13). Its contemporaneous FULL control resolves 4/6 in 62 calls, but the Django-15277 submission ends in `patch_apply_failed`; the whole paired sequence is therefore a preserved reliability observation, not admissible positive evidence. E1 resolves 5/6 and loses no observed FULL success, but its 70 calls produce only 14.21% raw paired saving and add 22 calls over the four jointly successful issues. E2 uses 60 calls and sends 21.93% fewer tokens than the control, but swaps outcomes: it gains Django-15277 and loses the control’s Django-16145 success. The predeclared failure-aware charge is consequently zero.

Table 13: N=6 instruction-epoch ladder. “Gross” compares each policy with its own unpruned trajectory; “raw paired” uses the contemporaneous persistent-FULL token denominator; “aware” charges any lost control success as zero saving. No treatment qualifies.

Policy	Resolved	Calls	Sent tokens	Gross	Raw paired	Aware	Lost
Persistent FULL	4/6	62	830,776	0.00%	0.00%	0.00%	0
E1: keep one prior epoch	5/6	70	712,763	27.90%	14.21%	14.21%	0
E2: keep two prior epochs	4/6	60	648,571	16.48%	21.93%	0.00%	1

The ladder also identifies backend variance directly. E1 and E2 remove nothing on issue 2: the first request is byte-identical to FULL, all three runs use temperature zero, top- p one, seed zero, and 100% realized retention, yet FULL’s first command differs. E1 and E2 match for six actions, diverge at action seven, and reconverge at action eight; both solve. Thus action divergence before any exclusion cannot be attributed to memory selection. The released reducer reports identical-input divergence and selection-active divergence separately and marks the ladder points inadmissible in the curves.

A targeted issue-5 diagnosis freezes E2’s successful four-issue prefix, resets the Django workspace for every fork, and first repeats only the next request. E2 retains 11,918 of 15,861 content tokens (24.86% first-request saving) and emits the same command in 5/5 repeats. FULL emits two commands in five byte-identical repeats. Restoring retired epoch 1 retains 13,945 tokens and reproduces one FULL command exactly; restoring epoch 2 retains 13,834 and produces a third command. All four commands are semantically equivalent searches for Django’s `runserver` implementation. The first action therefore does not implicate either retired epoch.

We then repeat the complete autonomous task from that same prefix and base workspace. Two E2 forks both resolve officially in 13 calls with the identical patch. Each sends 188,720 tokens versus its 239,979-token own-trajectory FULL counterfactual (21.36% gross saving). Two FULL forks both fail official grading, in 8 and 18 calls, with different incorrect patches. Mean calls are equal at 13 and

E2 sends 21.92% fewer tokens than the mean FULL execution (188,720 versus 241,687). This reversal does not show that pruning improves accuracy: it shows that the ladder’s single E2 loss is not a reproducible deterministic selection failure. E2 remains below the 30% target, and reliable promotion now requires randomized, interleaved repeats over complete ordered sequences. The complete ladder and diagnostic are released under `autonomous_persistent_instruction_epoch_n6_ladder_v1`.

Before spending another autonomous sequence, we screen hybrid floors on the 62 locked FULL requests (Figure 3). All user instructions and the active epoch remain whole. E0 offers 48.22% fixed-trajectory gross saving at $N=6$; one clean protocol turn per older epoch (P1) leaves 44.87%; one mutation, verification, and protocol turn per older epoch (M1/V1/P1) leaves 39.81%. Adding one or two ordinary recent turns per older epoch yields 37.43% and 35.61%, respectively. Keeping the most recent old epoch whole (E1) yields 31.80%. Thus the desired 30–50% opportunity band is not the remaining problem. The next autonomous candidate is E0+M1/V1/P1, with R1/M1/V1/P1 as its conservative fallback; whether either avoids E0’s extra-call cost is the next gate. Targeted pilots on the frozen issue-5 prefix reject both. M1/V1/P1 fails official grading after 18 calls while saving 34.97% on its own trajectory; R1/M1/V1/P1 fails in 9 calls while saving 37.23%. Both independently produce the same incorrect patch, mapping address 0 to Django’s default address instead of 0.0.0.0. Because E2 resolves 2/2 from the same prefix, one old mutation, verification, and protocol turn per epoch does not preserve all useful conditioning carried by two complete epochs. Neither hybrid advances.

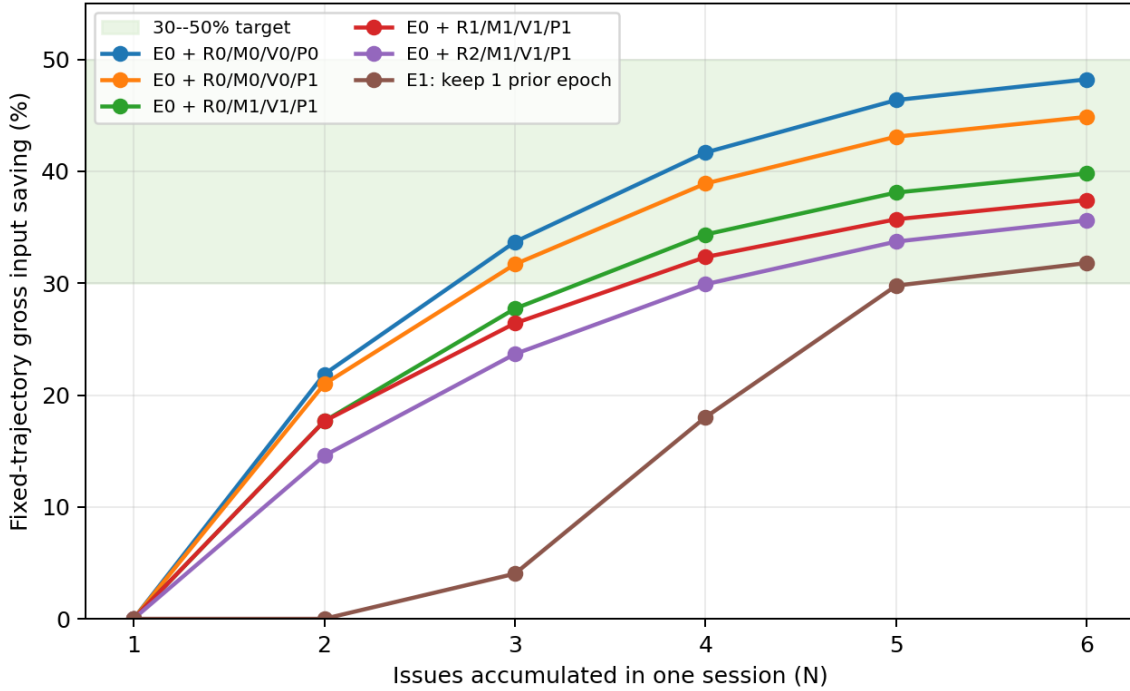


Figure 3: Fixed-trajectory logical opportunity as independent issues accumulate in one boundary-free session. The shaded region is the 30–50% target. The curves freeze model actions and therefore do not measure accuracy or calls-to-solution.

We also ran a structural-only opportunity screen for H1–H4 over the 23 growing prefixes of the first instrumented trajectory, with $h = 0$, two tail turns pinned, and a whitespace counter. After making explicit truncation receipts fail closed, H1 with $K_f = 0$ retires 938 cumulative whitespace

tokens (1.0%); delaying to $K_f = 4$ reduces this to 670 (0.7%). Version-guarded H2a retires 308 (0.3%), and H3 with $K_r = 1$ retires 177 (0.2%); $K_r \geq 2$ finds no opportunity on this trace. Guarded H2b correctly abstains because the captured verification lacks an explicit resource dependency. H4 with $K_x = 2$ has the largest opportunity, 3,180 tokens (3.5%), while $K_x \geq 3$ abstains after dependency pins. H1+H3 removes 1,115 (1.2%), and all guarded rules plus H4 at $K_x = 4$ remove 1,423 (1.6%). These are cumulative prefix occurrences rather than unique records, and they measure neither tokenizer-exact saving, next-action quality, nor task success. Their purpose is to confirm that the arms fire differently before spending model calls; the frozen gate and first autonomous pilot follow below.

Repeating the screen with the pinned Qwen3-Coder tokenizer changes the counts but not the ordering. Across 199,109 cumulative full-history tokens, H1 at $K_f = 0$ removes 2,660 (1.34%), H2a removes 833 (0.42%), H3 at $K_r = 1$ removes 603 (0.30%), and guarded H2b abstains. Aggressive H4 at $K_x = 2$ removes 11,370 (5.71%), whereas $K_x \geq 3$ abstains under the dependency pins. These are still opportunity counts, not behavior results.

8.8 Long-horizon, high-retention opportunity screen

We next applied the expanded grid to two officially resolved ordinary-text FULL trajectories: `django-15277` (26 decisions) and `django-15368` (30 decisions). Together their growing prefixes contain 284,848 exact Qwen3-Coder tokens; the decision-20-and-later suffix contains 127,332. Table 14 reports token-removal opportunity, not model behavior or task quality.

Table 14: Exact-tokenizer saving on long successful FULL trajectories. “Late” integrates only decision 20 onward.

Treatment	Whole run	Late suffix
H3, $K_r = 1$	6.23%	8.04%
H3, $K_r = 2$	0.73%	1.63%
H1, $K_f = 8$ + H3, $K_r = 2$	1.24%	2.78%
H1, $K_f = 12$ + H3, $K_r = 2$	0.83%	1.86%

The effect is phase dependent: every displayed treatment saves a larger fraction late than over the full trajectory. The conservative $K_f = 8, K_r = 2$ combination retains 98.76% overall but 97.22% in the late suffix, providing a high-retention candidate that fires on both task identities. The mechanism differs by task: H1 supplies the removals on `django-15277`, whereas H3 supplies them on `django-15368`. More history is not automatically better, however: $K_r \geq 4$ and $K_x \geq 6$ abstain on both traces, while immediate H3 and small working sets remove substantially more and remain higher-risk arms.

A blind fixed tail does not provide a stable retention contract. From decision 20 onward, keeping the first turn plus the last 20 complete turns retains 95.77% on `django-15277` but only 86.38% on `django-15368`; shorter fixed tails diverge further. Therefore the behavioral experiment will use an exact per-decision matched-token tail, not a shared nominal K . The long-horizon structural and high-retention JSON artifacts in the shared results directory preserve all cells and suffix aggregates.

8.8.1 Frozen quality at matched high retention

We screened the $K_f = 8, K_r = 2$ combination on the same two late suffixes using direct MLX and its 4-bit Qwen3-Coder-30B weights. This runtime is a logical ordinary-text consumer here; no engine-performance result is inferred. Two independent FULL runs reproduce all 18 late responses

and commands exactly. We rejected the original Ollama endpoint for this comparison: its first FULL replay matched all 30 historical responses, but an identical late repeat matched only 4/11 contents and 7/11 commands.

Table 15 pairs the semantic arm with a blind token tail using its per-decision materialized-token ceiling. To obtain a close match while preserving role order, only the boundary tool observation may be trimmed; assistant actions remain whole. The tail underfills by 20 cumulative tokens on `django-15277` and 60 on `django-15368`—0.065% of the semantic arm’s pooled materialized count.

Table 15: Late-suffix direct-MLX frozen replay. Exact command is relative to a contemporaneous, repeat-qualified FULL response.

Task	Decisions	Semantic ret.	Semantic cmd.	Tail ret.	Tail cmd.
<code>django-15277</code>	7	97.12%	4/7	97.08%	6/7
<code>django-15368</code>	11	97.29%	7/11	97.21%	8/11
Pooled	18	97.22%	11/18	97.16%	14/18

The hypothesis receives only partial support. Late exclusion opportunity is indeed larger, but the current whole-group semantics do not exploit it better than age alone. Pooled exact content is tied at 7/18, while the tail preserves 77.8% of commands versus 61.1% for H1+H3 despite removing 80 more cumulative tokens. On `django-15277`, delayed H1 retires two search maps and the assistant reasoning attached to them; the semantic arm records three immediate reacquisitions, one policy-excess reacquisition, and one invalid action. On `django-15368`, H3 retires one older same-file read and first changes a command at decision 23, two decisions before the tail.

This identifies a more precise next treatment. State authority and reasoning provenance must be separated: retain the assistant action/progress record, but replace a superseded tool observation with a typed resource/version/span stub. For discovery, preserve a compact unresolved-resource map rather than deleting the search bundle. The observation-only variants must again be compared with token tail at matched materialized counts before any autonomous rollout.

8.9 First instrumented frozen cohort

We then executed the ordered frozen gate on `scikit-learn__scikit-learn-13135`. The reference is a successful 23-decision ordinary-text mini-swe-agent trajectory using `qwen3-coder:30b`, its pinned tokenizer, temperature zero, and restored decision-point workspaces. This is next-action evidence, not yet an autonomous policy success rate. Table 16 compares each arm directly with a contemporaneous seeded FULL replay. Temperature zero alone was insufficient: an unseeded repeat diverged at decision 3 despite identical history through decision 20, so the original unseeded policy matrix is quarantined rather than interpreted as selection evidence. With seed zero, independent FULL-A and FULL-B runs match all 23 generated contents byte-for-byte. Exact-command and conservative-action rates use the 22 decisions on which FULL emitted a command; FULL’s decision 22 is itself format-invalid.

Table 16: First instrumented frozen cohort. Saving is cumulative over growing decision prefixes. “Final” means equality of the frozen decision-23 action, not autonomous SWE-bench resolution.

Treatment	Mean ret.	Saving	Exact text	Exact cmd.	Cons. action	First text div.	Final
FULL	100.0%	0.0%	100.0%	100.0%	100.0%	–	yes
FULL repeat	100.0%	0.0%	100.0%	100.0%	100.0%	–	yes
DAG-EXCLUDE@100	99.8%	0.3%	91.3%	95.5%	95.5%	21	yes
Matched token tail	99.8%	0.3%	91.3%	95.5%	95.5%	21	yes
H1 consumed search, $K_f = 0$	98.9%	1.3%	52.2%	59.1%	68.2%	10	yes
H1 consumed search, $K_f = 4$	99.2%	1.0%	65.2%	68.2%	72.7%	14	yes
H2a current-state read	99.7%	0.4%	73.9%	77.3%	77.3%	17	yes
H2b verified write	100.0%	0.0%	100.0%	100.0%	100.0%	–	yes
H3 same span, $K_r = 1$	99.8%	0.3%	91.3%	95.5%	95.5%	21	yes
H4 working set, $K_x = 2$	95.1%	5.7%	43.5%	54.5%	59.1%	9	yes
H1+H3	98.6%	1.6%	47.8%	54.5%	59.1%	10	yes
H2a+H3	99.4%	0.7%	73.9%	77.3%	77.3%	17	yes
All guarded, $K_x = 4$	98.3%	2.1%	52.2%	59.1%	63.6%	10	yes

Two mechanism findings are currently admissible. First, the instrumented trace supports one TRACE-EXACT-OPERATION-RESULT certificate: two complete reads execute the same command and return the same bytes for the same cwd, environment, and resource version despite different reasoning prose. Removing the older turn saves only 603 cumulative tokens. It changes decisions 21–22, from writing a temporary diff to displaying it and then another invalid response, but returns to the exact final submission action at decision 23. Operational proof and behavioral identity are therefore empirically distinct even in the strongest current DAG tier.

The new within-observation token-tail control materially matches the DAG arm: 198,500 versus 198,506 cumulative materialized tokens. Both first change the action at decision 21, preserve 95.5% conservative action agreement, and return to the exact final action. This one trace therefore does not yet show a quality advantage for certified exclusion over recency at equal materialized context.

Second, immediate H1 is too aggressive. Once a discovery result is consumed by a concrete read and followed by a non-read transition, it removes one 190-token causal turn from each later prefix. The first text change occurs at decision 10, the first command-string change at decision 12, and the first conservatively different shell action at decision 13. Across all prefixes it saves 2,660 tokens (1.34%) but preserves only 68.2% conservative action agreement, with six immediate reacquisitions and one policy-excess reacquisition. A search listing can remain a durable navigation map after one file is opened. Delaying retirement to $K_f = 4$ moves first conservative action divergence from decision 13 to 16 and raises action agreement only to 72.7%, while five immediate reacquisitions remain. Delay is therefore insufficient: the next H1 refinement must protect unresolved discovery branches or retain a compact resource-map tombstone.

We implemented the strict unresolved-branch variant separately. It requires a complete successful read/diff for every concrete resource returned by the search, followed by the transition and K_f delay. It abstains at every prefix for all $K_f \in \{0, 1, 2, 4\}$ on this trace because several discovered example and test-file branches were never consumed. This is the correct conservative outcome, but supplies no saving; a compact model-visible resource-map stub is the next discriminating treatment.

H2a also fails as a safe-exclusion claim. Retiring a 119-token successful-write turn after a complete same-version read/diff saves 833 cumulative tokens (0.42%), but first changes the shell action at decision 17, preserves only 77.3% conservative action agreement, and changes action validity at decision 20. The current bytes represent file state, but they do not necessarily carry the mutation’s intent or the agent’s progress state. A model-visible mutation tombstone and the stricter verified-write H2b arm must therefore be evaluated separately. H2b correctly abstains on this trace because the verification record lacks explicit dependency metadata; its 23 outputs exactly match

FULL. This is fail-closed correctness, not evidence that H2b can save context when a verification receipt is available.

On this trace H3 at $K_r = 1$ retires exactly the same older read as the trace-exact DAG certificate, so all selected histories and generated outcomes coincide: 603 cumulative tokens saved and 95.5% conservative action agreement. This does not establish the safety of H3’s weaker version/span criterion; a task containing non-identical overlapping reads is required to separate it from byte-identical duplicate elimination.

H4 at $K_x = 2$ reaches the largest saving, 11,370 cumulative tokens (5.71%), but changes the action as soon as it first fires at decision 9 and preserves only 59.1% conservative action agreement. Its zero immediate-reacquisition count is not exculpatory: frozen decisions do not compound, and the generated action can change without explicitly rereading the hidden resource. H4 remains an aggressive capacity baseline, not a safe rule.

The two-rule combinations show no positive interaction on this trace. H1+H3 saves 3,263 cumulative tokens (1.64%) but falls to 59.1% conservative action agreement, below H1 alone. H2a+H3 saves 1,436 (0.72%) and retains H2a’s 77.3% action agreement because H2a has already changed the late decisions on which H3 fires. The predeclared three-rule H1+H3+H2b arm is input-equivalent to H1+H3 here because H2b abstains; it is recorded as a deterministic equivalence, not charged as another model cohort.

Finally, the all-guarded treatment at $K_x = 4$ saves 4,096 cumulative tokens (2.06%) with 63.6% conservative action agreement. H2b and H4 abstain at that setting, so this is effectively H1+H2a+H3. Its slight recovery relative to H1+H3 despite removing more context demonstrates non-monotone model sensitivity, not a robust policy improvement.

8.10 Synthetic mechanism controls and structured result evidence

The preceding negative outcomes could arise from a policy weakness, a rule that does not implement its declared predicate, or an experiment-driver error. We therefore added three small mini-swe-agent-shaped diagnostic trajectories with predeclared dependency structure: (i) a search whose every returned branch is subsequently read before a non-read transition, (ii) a versioned write followed by two identical current-version reads and a resource-linked passing test, and (iii) a long test observation whose decisive assertion and source location lie away from both output boundaries. These are mechanism controls, not substitute software-engineering benchmarks.

All structural gates pass: strict H1 removes exactly the consumed discovery bundle; H2a/H2b remove exactly the witnessed write; H3 removes only the older same-version duplicate read; and every predeclared necessary fact remains in the selected history. The exercise also exposed and corrected an independent replay-driver bug: the maximum-decision limit was applied before the minimum decision, so selecting one late probe could silently execute zero calls.

We then queried one predeclared decision from each task using Ollama 0.30.8, Qwen3-14B, its matching tokenizer, temperature zero, seed zero, and a 1,024-token output ceiling. Contemporaneous FULL repeats reproduce all 3/3 contents and commands exactly. Table 17 reports candidate behavior against those repeats. H3 is exactly inert in the constructed same-version duplicate case. Strict H1 and verified-write H2b change the exact terminal command, but both changes are between harmless `echo` completion acknowledgements after the requested fix has passed; this is a concrete reason to report completion-state/action equivalence beside exact command identity.

Treatment	Retained	Exact command	Diagnostic action
Strict H1 consumed search	85.75%	no	equivalent completion no-op
H2b verified write	78.53%	no	equivalent completion no-op
H3 duplicate current read	84.18%	yes	exact FULL command
Fixed head/tail tool result	60.78%	no	wrong phase: diff before edit
Lexical matched-span result	80.02%	no	targeted edit, correct file
Structured-evidence result	66.88%	no	targeted edit, correct file

Table 17: Single-decision synthetic mechanism diagnostics. Retained is the tokenizer-measured materialized fraction of the full request. These rows test implementation and controlled sensitivity; they do not estimate autonomous task success.

The structured-evidence materializer is deliberately not another positional head/tail rule. It scores failure and traceback lines, source locations, diff headers and hunks, source symbols, command-resource identities, and current task/query matches; it preserves the return-code/output envelope and local context around selected anchors. If no positive structural evidence exists, it fails closed to the complete observation. In the long-result diagnostic, the tool observation contains 856 tokenizer tokens. Structured evidence removes 315 of 951 total request tokens (33.12%), versus 190 (19.98%) for lexical matching. Both produce a targeted edit on the correct file rather than the exact FULL command. Fixed head/tail loses the middle failure evidence and produces a premature diff action. The generated edits were not executed, so this is evidence about action phase and target rather than task correctness. It nevertheless supports carrying structured evidence forward as a candidate materializer, while whole-group H1/H2 retirement should next be compared with model-visible resource-map and mutation receipts.

8.11 Real oversized-observation materialization

We next froze the successful `scikit-learn-13135` trajectory and applied the same 512-token materialization threshold without removing any logical record. The trace contains a 2,526-token, 292-line source dump and later 685- and 737-token observations. A third current FULL repeat reproduces all 23 contents and all 22 defined commands byte-for-byte. This qualifies that control but, as the threshold-repeat audit below shows, does not establish universal endpoint determinism.

Across all 23 decisions, structured evidence reduces 199,109 cumulative materialized tokens to 188,072, a 5.54% saving. It preserves 10/22 exact commands and 12/22 conservatively equivalent shell actions; its 22/23 format-valid decisions equal FULL’s validity count. Thus selective result evidence is more conservative than deleting causal groups, but it is not behavior preserving even though every logical record remains selected.

An adaptive 256-token threshold repeat falls in the same effective materialization class as 512 on this trajectory: both retain exactly 188,072 tokens at the same 23 decisions. The only additional record entering the structured path at 256 has 259 tokens, and all of its lines are retained, so the byte-identity no-op rule restores the canonical observation. Nevertheless, the 256 repeat preserves 14/22 conservative actions versus 12/22 at 512 (and 12/22 versus 10/22 exact commands). This is same-input endpoint variability, not a threshold effect. We therefore stop the remaining within-class threshold sweep and require future curves to use observation-size breakpoints plus an explicit model-visible request digest. A nominal parameter grid is not a quality-saving curve when multiple settings materialize identical requests.

Table 18 holds decisions 3–10 fixed to compare the three result materializers on identical prefixes. Structured evidence gives up most of the attainable compression but doubles exact/conservative

action agreement relative to matched-span and quadruples it relative to fixed head/tail. All candidate actions are format-valid. Because these are frozen next actions over a successful source trajectory, not executed rollouts, the table establishes a compression–agreement gradient rather than autonomous task utility.

Materializer	Materialized saving	Exact command	Conservative action
Structured evidence	4,335/57,480 (7.54%)	4/8	4/8
Lexical matched span	16,248/57,480 (28.27%)	2/8	2/8
Fixed head/tail	17,406/57,480 (30.28%)	1/8	1/8

Table 18: Frozen real-trajectory comparison on decisions 3–10. Logical record retention is 100% in every arm; saving is solely within oversized tool observations.

The trace audit also supplies an important negative control. The two long Django cohorts contain, respectively, zero oversized observations and one 565-token observation for which the structured selector keeps every line. An initial nominal no-op nevertheless changed 2/11 commands because the harness reconstructed the complete observation with `splitlines()` and join, dropping trailing-byte detail while reporting identical token counts. That artifact is quarantined. Complete-line selection now returns the canonical record bytes, and a byte-identity regression test prevents recurrence. This is the text-materialization analogue of the cache-identity rule: zero measured token change is not evidence of an identical model input.

Figure 4 collects the repeat-qualified frozen comparisons without pooling incomparable decision cohorts. It is intentionally labelled as a proxy frontier: only autonomous official-resolution experiments populate the accuracy–saving curve. The figure makes the present screening decision visible—whole-group heuristics lose agreement rapidly, matched recency is a strong control, and structured materialization is the only current candidate admitted to a bounded task-utility probe.

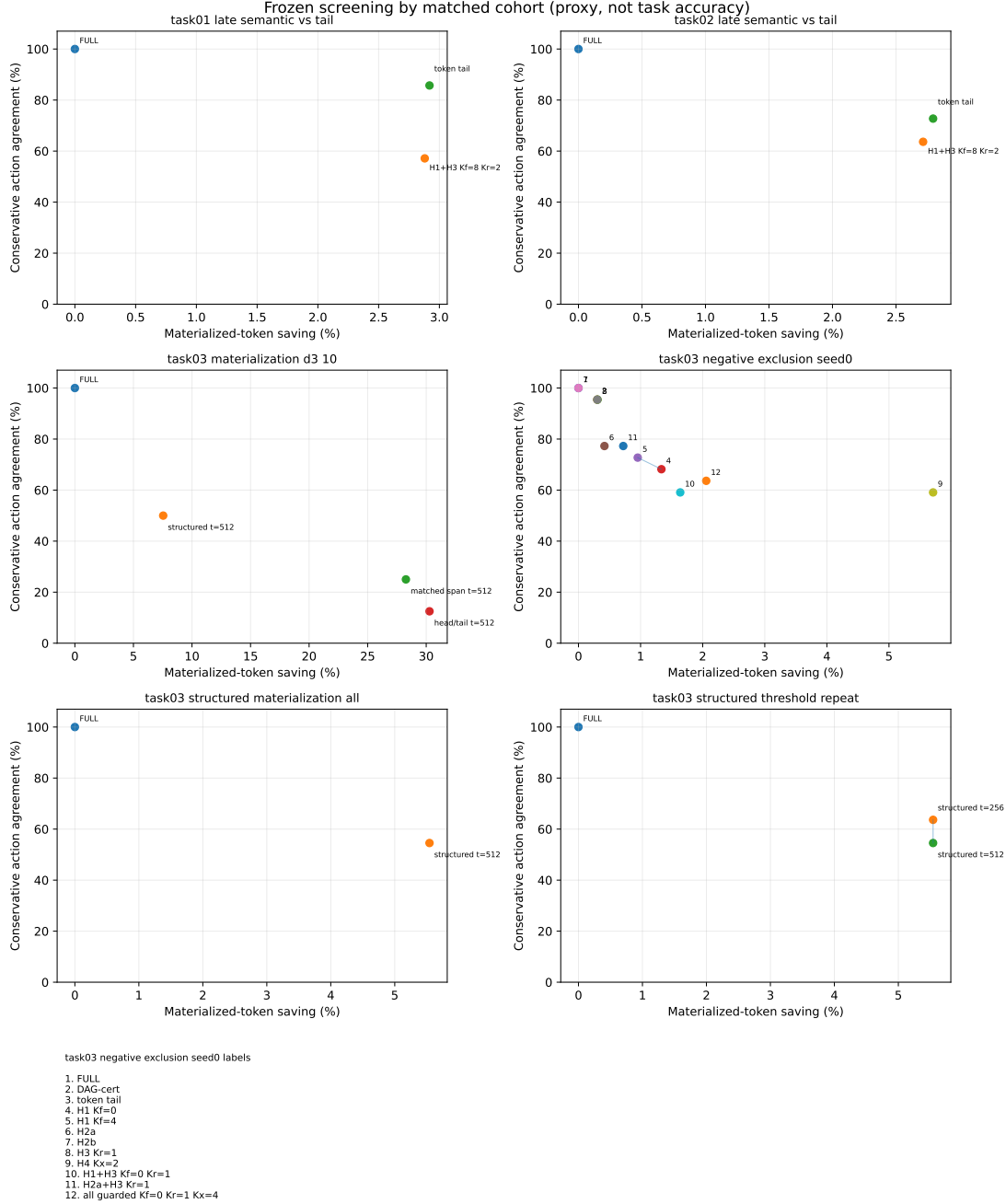


Figure 4: Materialized-token saving versus conservative next-action agreement, separated by matched frozen cohort. Agreement is diagnostic and is not task accuracy.

8.12 Autonomous structured-observation pilot

We then ran the structured materializer autonomously on `scikit-learn-13135`, still retaining every logical causal group. The 512-token treatment resolves officially in 22 calls and materializes

159,482 of 169,850 trajectory-counterfactual message tokens, a 6.10% gross saving. A successful FULL control resolves in 23 calls and consumes 197,796 tokens. A second FULL execution stops after 19 calls with a nonempty 778-byte terminal workspace patch but submits an empty patch, so its primary official outcome is failure. Thus the treatment is a promising one-task point, not an accuracy estimate or a paired trajectory-saving claim: the two FULL trajectories have different lengths and outcomes. It advances only to repeated same-host evaluation on multiple task identities. The compact ledger is released under `autonomous_structured_task5_v1`.

8.13 Model-visible resource and mutation receipts

Binary H1/H2 deletion removes both payload and progress evidence. We therefore implemented a separate *observation-receipt* realization: retain the historical assistant action verbatim and replace only its paired tool observation with a compact resource-map or mutation receipt derived from the current prefix. Receipt and source tokens are measured with the active model tokenizer. The treatment emits only when the receipt is strictly smaller; otherwise it restores the complete canonical group. Malformed pairs and prefix-invalid witnesses fail closed under a distinct reason code.

This size gate was required empirically. The first general JSON receipt expanded the short synthetic H1 observation from 26 to 134 tokens and H2b from 15 to 133. It increased total context and made the H1 response format-invalid. After compacting the syntax and adding the gate, the candidate receipts are 43 and 46 tokens, respectively, so both short controls abstain. Their serialized requests are byte-identical to FULL, and both contents and commands reproduce exactly. A receipt system must count its own disclosure cost; a semantically sensible summary is not a saving when it exceeds its source.

To verify activation independently of this size boundary, we derived two synthetic mechanism controls by appending 96 deterministic, semantically irrelevant diagnostic lines only to the old H1 search observation or H2b mutation observation. Table 19 shows that the receipt path then removes most payload while preserving the old action and a model-visible state fact. Every candidate emits a valid terminal completion action, and role alternation, prefix-local witnesses, and absence of future message leakage pass audit. H1’s identical-input FULL repeats differ in terminal wording, so its exact-command difference is not attributable. H2b’s FULL repeats are exact, but its single receipt and deletion outputs both differ only in terminal `echo` wording. These rows qualify the mechanism, not autonomous utility.

Synthetic treatment	Full/materialized	Saving	Action phase
H1 whole-group drop	1,449/337	76.7%	terminal completion
H1 observation receipt	1,449/410	71.7%	terminal completion
H2b whole-group drop	1,410/278	80.3%	terminal completion
H2b observation receipt	1,410/385	72.7%	terminal completion

Table 19: Size-gated receipt activation controls. H1 replaces a 1,082-token observation with 43 tokens; H2b replaces 1,071 with 46. The receipt costs more than deletion because it retains the original assistant action.

The next receipt arm should remain separately labelled. A *witness-attached receipt* would remove the old pair and append a compact fact to the already selected downstream H1 read, H2a current-version read/diff, or H2b verifier. It can save small 44–49-token groups without introducing an extra role pair, but it moves evidence later and discards the original action/progress text. It therefore requires its own frozen replay and autonomous qualification rather than being folded into

observation-receipt.

We then applied the portable v4 policy to decisions 22–24 of the successful real Django task 15277 trajectory. Two contemporaneous FULL controls agree on two of three exact next commands and first differ only at decision 24. Two semantic-receipt repeats produce byte-identical model output at all three decisions, but agree with the first FULL control on only one of three commands and first differ at decision 22. The treatment materializes 20,893 of 21,907 content tokens, a 4.63% saving, by replacing three older same-version reads at each decision. This is a bounded action probe, not task-accuracy evidence. It nevertheless shows why selection evidence and presentation must be separated: the phrase “older read superseded” is model-visible text and can itself prime a consolidation action even though the newer source evidence remains selected.

The default portable realization therefore keeps rule, resource, version, completeness, and witness evidence entirely in out-of-band request metadata. If the chat protocol requires a paired observation, it emits only a role-valid empty-result stub. Metadata has no prompt-token cost; only the stub is counted in materialized tokens. Rich semantic receipts remain an explicitly labelled presentation treatment and require their own quality curve. The next frozen arm compares this metadata-only stub with the same contemporaneous FULL controls before any autonomous expansion. In that arm the stub materializes 20,503 tokens, a 6.41% saving. Its two repeats have identical request hashes but agree with each other on only one of three exact commands; against the first FULL control they agree on one of three and zero of three, respectively, and both first differ at decision 22. All generated actions remain in the late verification/submission phase. Thus removing the semantic phrase improves savings but does not restore exact trajectory identity. This rules out receipt wording as the sole cause, while leaving task quality unresolved until autonomous execution.

8.14 Predeclared autonomous quality-saving campaign

The next campaign locks the five longest tasks in the Easy-14 baseline-success stratum before observing any treatment outcome: Django 15277, pytest 7982, Django 15741, Sphinx 8721, and scikit-learn 13135. Their admitted no-PRA trajectories contain 16–33 calls and 123,003–287,951 cumulative prompt tokens. This is a conditional preservation cohort, selected for policy development; it is not an unconditional SWE-bench accuracy sample.

Each task first receives two FULL controls. A certificate-only DAG arm at a 100% ceiling measures safe operational-exclusion yield and stops after two tasks if mean saving is below 2%. The main matched-budget factorial then contains only three compressed presentations: token/causal tail, whole-record H1+H3 retirement, and the same H1+H3 decisions with the assistant action/provenance retained and only the observation payload replaced by a typed resource/version/witness stub. The first autonomous target is 90%; 80% is considered only after a 90% condition preserves quality. The common configuration protects two initial and four recent complete turns and uses $K_f = 8$, $K_w = 1$, and $K_r = 2$. Nominal retention is not plotted: every point uses realized model-visible tokens. The frozen generation contract also fixes a 1,024-token completion ceiling. The campaign runner now rejects an absent or non-positive ceiling before launching: an uncapped response can run away, altering both agent behavior and end-to-end token cost rather than constituting a comparable policy observation. This guard was added after an initial long-five launch omitted the ceiling: four completed controls from that launch and a third-task response that ran past 10,774 generated tokens are quarantined in full and contribute no paper outcome. The replacement campaign has a new identity and reruns every control under the corrected ceiling. Selector-only corrections may reuse those completed controls through a hash-bound import. New runs use a schema-2 pairing contract that requires a shared non-null pair identifier, immutable container image identity, dataset/task identity, tokenizer and model revisions, decoding fields, harness and grader versions, scaffold identity,

instrumentation mode, and a normalized agent-behavior digest. Null equality is not evidence. Historical schema-1 pairs require an explicit, reasoned legacy exception and emit their missing identity fields in the reduced artifact. Recorded identity mismatches other than the narrowly declared repository revision remain fatal.

The primary ordinate is official task resolution. The abscissae separately report within-run materialized-history saving, paired trajectory message-input saving (therefore including additional calls), and a failure-aware coordinate. A failed candidate is charged at least its paired FULL workload so early termination cannot masquerade as efficiency. Calls-to-solution, direct reacquisition tokens, calls-to-solution, and first action divergence remain secondary diagnostics. Endpoint-reported completion tokens form an additional total-model-token coordinate only when all calls provide usage. Adaptive gates operate on distinct task identities: repeats collapse within task before mean saving and resolution are computed, so repeated failures of one task cannot satisfy a multi-task boundary. The campaign stops an arm after two distinct task failures, after less than 2% task-macro gross saving on two tasks, or after task-macro resolution falls below 80% on at least three tasks. Only the better of whole-record retirement and payload-only retirement competes with matched recency beyond the first three tasks. No lexical, dense, learned, second-model, or second-harness search begins before this policy freeze.

Failures enter a frozen, one-causal-group-at-a-time oracle add-back audit at the first command divergence. Each counterfactual restores a complete action-observation group while holding the selected request and reference fixed otherwise; candidate groups are tried from the fewest restored tokens upward. Recovery identifies a sufficient omitted group for diagnosis, not a deployable policy or a proof that the group was uniquely necessary. The autonomous accuracy curve therefore never includes oracle-assisted outcomes.

8.15 First autonomous qualification

We next ran an autonomous pilot on `django_django-15277` from fresh SWE-bench workspaces. All executions use `mini-swe-agent 2.4.6`, `qwen3-coder:30b`, the pinned Qwen tokenizer, temperature zero, $\text{top-}p = 1$, seed zero, whole-record text materialization, and the official SWE-bench Docker grader. Earlier pilot attempts that were no-ops or otherwise invalid were quarantined and do not enter Table 20. The corrected proxy forwards the incoming request exactly while a negative policy is a logical no-op; the included H3 run records five such exact decisions before its first exclusion. The primary endpoint is the official grade of the model patch submitted through the normal agent protocol. We additionally report a post-hoc official grade of a provenance-checked patch extracted from terminal tracked-workspace state. This auxiliary outcome diagnoses solution state; it never replaces or relabels the primary submission outcome. Extraction fails closed if untracked state makes a tracked-only patch incomplete.

Table 20: First autonomous single-task pilot. Token totals are cumulative message-content tokens and exclude chat-template tokens. Because the rollouts diverge, their full-history totals and call counts are not paired prefixes. H3’s lower call count is failure termination, not fewer calls to solution. “Aux.” is the separately labelled final-workspace outcome.

Policy	Primary	Aux.	Calls	Full tok.	Selected tok.	Saving	Reacq.
FULL-A	1/1	unavailable	26	131,530	131,530	0.00%	0
FULL-B	0/1	1/1	29	154,173	154,173	0.00%	0
H3, $K_r = 1$	0/1	0/1	19	91,520	85,640	6.43%	6

FULL-A passes the primary official grader. Its auxiliary extraction is unavailable because the

terminal checkpoint includes untracked `patch.txt`; rejecting an incomplete tracked-only reconstruction is the required fail-closed outcome. FULL-B makes the intended code change but diverges from FULL-A at call 9 and finally prints a source excerpt rather than a unified diff, so its primary submission is rejected. The provenance-checked 749-byte auxiliary workspace patch (SHA-256 prefix 01522b) resolves officially. Thus FULL-B’s primary failure is submission protocol, not solution state.

The primary FULL submission endpoint remains 1/2 on this task, while the available primary and auxiliary evidence establishes resolving code states for both executions. The trajectories still vary materially in calls and actions. One H3 failure cannot estimate a population success-rate delta. A separate fixed first-request probe reproduces assistant content and command exactly in 3/3 calls; this does not qualify a growing autonomous trajectory, where generated reasoning and environment observations enter later requests.

The qualified H3 rollout has the same model-visible request as FULL-B through call 5. Call 6 is the first actual exclusion: assistant reasoning changes immediately but its shell command remains the same. The shell command first changes at call 7. This is a causal next-action divergence, not by itself proof that selection caused the eventual official failure. The subsequent trajectory records six reacquisitions of excluded resources. It eventually applies an unscoped global `sed` replacement: the intended guard appears in `CharField`, but the already guarded `BinaryField` is also rewritten, producing a malformed duplicate `if`. The final submission command prints broad `grep` context rather than a Git diff; consequently, the submitted `model_patch` is not a diff and the primary grader reports a patch-application error. Unlike FULL-B, the H3 workspace is not a resolving solution: its 1,090-byte auxiliary patch (SHA-256 prefix b1171e) contains the malformed duplicate guard and also scores 0/1.

The scientific conclusion is deliberately narrow. H3’s same-version/span condition is not behaviorally inert: a newer source read can supersede exposed bytes without proving the older record’s localization, reasoning, or mutation intent irrelevant. The model-visible H3 trajectory and failure mechanism were reproduced exactly in a second execution after the no-op request fix, but only the corrected execution enters the table. The auxiliary comparison separates the observed failure mechanisms: FULL-B reaches a resolving state but violates the submission protocol, whereas H3 fails under both outcome definitions. This single task still does not estimate a population H3 penalty. Frozen replay remains a screening tool, while autonomous claims require repeated FULL controls and multiple task identities. H3 is explicitly a heuristic arm; certificate-backed and strict exclusions continue to fail closed when their evidence predicates are not met. The next H3 treatment should preserve a compact model-visible source/progress tombstone or require stronger evidence than shared resource-version and span.

8.16 FULL-control replication on a second task

We repeated the two-control procedure on `django_django-15368`. The two executions use the same pinned model, tokenizer and decoding fields as Task 01, fresh containers from the same image, and identical initial workspace and environment fingerprints. All 43 model requests are exact FULL pass-through and no request has an upstream error. Table 21 summarizes the result.

Table 21: Autonomous FULL-control replication across task identities. P is the primary submission outcome and A is the auxiliary final-workspace outcome. Calls are shown in parentheses. First command divergence compares the two FULL controls within a task; it is not a policy divergence.

Task	A:P	A:A	B:P	B:A	First cmd. div.	Heuristic
Task 01	1/1 (26)	unavailable	0/1 (29)	1/1	9	H3 pilot
Task 02	1/1 (30)	unavailable	0/1 (13)	1/1	6	none

Task 02 FULL-A resolves officially. FULL-B submits a source excerpt instead of a unified diff and receives a primary official patch-application error. The provenance-checked 731-byte auxiliary workspace patch (SHA-256 prefix `f5d867`) resolves 1/1. As on Task 01, FULL-B therefore reaches a resolving solution state but fails the submission protocol. FULL-A’s auxiliary extraction is unavailable because untracked `bulk_update_fix.patch` and `test_fix.py` make a tracked-only reconstruction incomplete.

The Task 02 request sequence is identical through call 5. On that identical call-5 request, the assistant content differs while the Bash command remains equal; the accumulated request and command first differ at call 6. The auxiliary success narrows the failure to submission behavior, but does not remove the material instability in reasoning, calls, and actions. Temperature zero and a fixed seed cannot be treated as a guarantee of repeatable generation or agent protocol adherence for this endpoint.

The predeclared Task 02 gate required both FULL controls to resolve with sufficiently stable trajectories before testing strict H1. That primary gate fails, so we run no Task 02 H1, H2a, H3, or combined policy. Across two task identities, the primary FULL submission resolves exactly one of two controls on each, while auxiliary grading shows that both FULL-B workspaces resolve. This replicated protocol and trajectory instability strengthens the need for repeated FULL controls, but four runs are not a population estimate of model or agent reliability.

The auditable artifacts are under `docs/papers/shared/results/paper8.5_agent_memory/autonomous_task01_qwen30` and `autonomous_task02_qwen30`; they include per-request selection receipts, full trajectories, token counters, primary submission outcomes, separately labelled auxiliary workspace outcomes, and frozen run provenance.

9 Semantic-Role Ablations

For the typed progress spine, we remove one role at a time:

–SOURCE, –MUTATION, –VERIFICATION, –PROGRESS, –RECENT.

We measure official success, calls, rereads, repeated tests, and first-divergence rate. These ablations test which semantic roles carry long-lived task state.

10 Oracle Utility

For a causal bundle g and future decision point t , define

$$U(g, t) = \text{Quality}(H_t) - \text{Quality}(H_t \setminus g).$$

The first implementation uses action-level behavioral outcomes rather than token-level attribution. Its future reference action is available only to the offline scorer and is never serialized into model input.

Bundles are analyzed in the hierarchy

record type $\rightarrow$ causal bundle $\rightarrow$ source span.

The oracle provides an upper bound on how compact agent history could be if future utility were known.

We distinguish two oracle levels. The *next-action oracle* fixes system/task/head/tail state and searches subsets of middle causal bundles, scoring exact command, semantic command equivalence, target file, and edit intent. Leave-one-bundle-out and role ablations provide the first utility labels; pairwise ablations and bounded beam or branch-and-bound search address non-additive bundle interactions before a subset is called minimal. The more expensive *rollout oracle* resumes a small predeclared set of decision points from exact workspace snapshots and scores eventual official success, calls, and recovery behavior. It tests whether next-action agreement is an adequate proxy for task utility.

We also express the oracle in the dual exclusion form. Holding protected state fixed, it searches for the largest set of middle bundles whose removal preserves the scoring outcome. This upper bound is compared with DAG-certified exclusions using behavioral precision (how often a certified exclusion survives the counterfactual), removal recall relative to the oracle-removable set, and token coverage. “Never used again” in the observed path is not an oracle label; utility must be established by counterfactual generation or rollout.

10.1 Three-trajectory omission oracle

We executed the next-action oracle at one late decision in three previously repeat-qualified successful trajectories using the same 4-bit Qwen3-Coder-30B model and tokenizer, temperature zero, seed zero, and a direct MLX endpoint. The decisions contain 7,599, 6,597, and 11,111 model-visible history tokens. System state, the task record, the protected head, and the recent tail remain fixed; each singleton trial removes exactly one complete middle causal bundle. Pair and bounded beam trials are generated only from surviving parents and are capped at eight trials at depth two and four trials at later depths.

Exact command identity and semantic transition equivalence are reported separately. The latter requires a valid action with the same typed operation and target-resource set; for example, `grep` and `sed` reads of the same modified source file are equivalent at this level. It is deliberately weaker than exact replay and is not a task-success label.

Table 22: Offline omission-oracle headroom at one late decision per trajectory. Singleton columns are survivor counts. Best columns report the largest tested omitted-token fraction preserving the indicated criterion at any bounded-search depth. Oracle outcomes are not deployable-policy accuracy.

Task	Tokens	Singletons	Exact	Semantic	Best exact	Best semantic
django-15277	7,599	20	2	14	24.28% (d4)	27.04% (d5)
django-15368	6,597	21	3	14	4.00% (d1)	24.53% (d4)
sklearn-13135	11,111	16	0	16	0.00%	38.16% (d3)

The result is strongly non-additive and sometimes non-monotone. On django-15277, the pair formed from the only two exact-safe singleton omissions does not remain exact. Conversely, four of eight pairs proposed by the semantic beam recover exact command identity, and one four-bundle subset removes 24.28% while remaining exact. At depth five exact identity disappears again although

one 27.04%-saving subset retains the semantic transition. On django-15368 none of the three exact-safe singleton pairs stays exact, while semantic equivalence survives all four tested depth-three and depth-four subsets. Greedily summing singleton savings is therefore invalid; even this small oracle needs explicit subset evaluation.

Fresh FULL probes also reveal why exact strings cannot be the utility axis. The original-position plan and materialized-token count are unchanged, but only django-15277 reproduces its pinned command exactly after the server restart. The other two FULL probes choose different commands that perform the same typed operation on the same resource. Exact trajectory remains a useful implementation and repeatability diagnostic; semantic transition and eventual autonomous success remain distinct quality outcomes.

10.2 Matched retirement factorial

At the same three decisions we compare H1+H3 whole-bundle retirement, the identical negative decisions realized as payload receipts, and token-tail recency constrained not to exceed each structured arm’s realized token count. Table 23 scores semantic transition against the pinned FULL action and exact command against a contemporaneous FULL probe. Hard-ceiling recency can round below the structured arm when the next retained unit does not fit; this is recorded as unused budget rather than hidden.

Table 23: Frozen retirement factorial. Savings are materialized-token reduction for Tasks 1/2/3. Sem. and Exact are survivor counts out of three decisions.

Arm	T1 save	T2 save	T3 save	Sem.	Exact
Whole H1+H3	1.24%	3.14%	0.00%	3/3	1/3
Payload receipt H1+H3	0.00%	1.44%	0.00%	2/3	2/3
Matched tail $\leq$ whole	3.05%	5.96%	0.00%	3/3	1/3
Matched tail $\leq$ receipt	0.00%	2.90%	0.00%	3/3	2/3

The payload-receipt arm does not earn a default-policy promotion. Its tokenizer gate correctly abstains whenever a receipt would not be smaller than the source, leaving Tasks 1 and 3 at zero saving; on Task 2 it changes the semantic transition at only 1.44% saving. Whole-record H1+H3 averages 1.46% saving, below the predeclared 2% yield gate. The recency control constrained by the whole-record arm averages 3.00%, preserves all three semantic transitions, and is the only frozen candidate advanced to repeated autonomous qualification. These are three decision probes, not an accuracy estimate; official task success and calls-to-solution are required before profile promotion.

The digest-bound trial ledgers, queues, reductions, and per-arm requests are under `docs/papers/shared/results/paper8_5_agent_memory/`, in `oracle_headroom_v1` and `retirement_factorial_v1`.

10.3 Repeated matched-recency qualification and stop decision

We next executed the only nondominated frozen candidate, matched token-tail with two protected head turns, four protected tail turns, and a nominal 95% ceiling. An initial cross-host pilot was not pairable: the first `find|grep|head` observation returned identical paths in different filesystem order, and commands diverged before selection activated. We therefore backfilled FULL controls on the same agent/Docker host as each treatment and retained the cross-host rows only as audit evidence.

Table 24 reports the qualified cohort. Each task has two FULL and two treatment runs under the same model digest, tokenizer, generation fields, agent behavior, image, and host. Mandatory system, task, and incomplete current-turn state may exceed the fractional ceiling. Whole-record

packing can also round below it: the realized treatment savings range from 4.86% to 10.17%, rather than being forced to exactly 5%.

Table 24: Repeated same-host autonomous qualification. Calls list the two runs. Saving pools message-content tokens within the treatment for each task.

Task	FULL primary	Tail primary	FULL calls	Tail calls	Tail save
django-15277	1/2	2/2	40, 27	31, 15	6.05%
pytest-7982	2/2	2/2	21, 23	12, 17	10.17%
scikit-learn-13135	2/2	0/2	20, 23	19, 19	4.86%
Three-task aggregate	5/6	4/6	25.67 mean	18.83 mean	6.35%

The policy passes the 2% yield gate but fails the predeclared quality gate: task-macro primary resolution is 83.3% for FULL and 66.7% for treatment, below the required 80%. It is therefore stopped before more tasks, lower budgets, model families, or agents. The treatment’s lower mean call count is diagnostic rather than a causal efficiency claim. Even the FULL repeats first diverge at calls 2, 4, and 3 on the three tasks, so temperature zero does not make an autonomous trajectory deterministic. Failure-aware paired accounting charges a failed treatment at least the corresponding FULL workload; the two short Task 5 failures consequently earn zero paired saving rather than a false efficiency benefit.

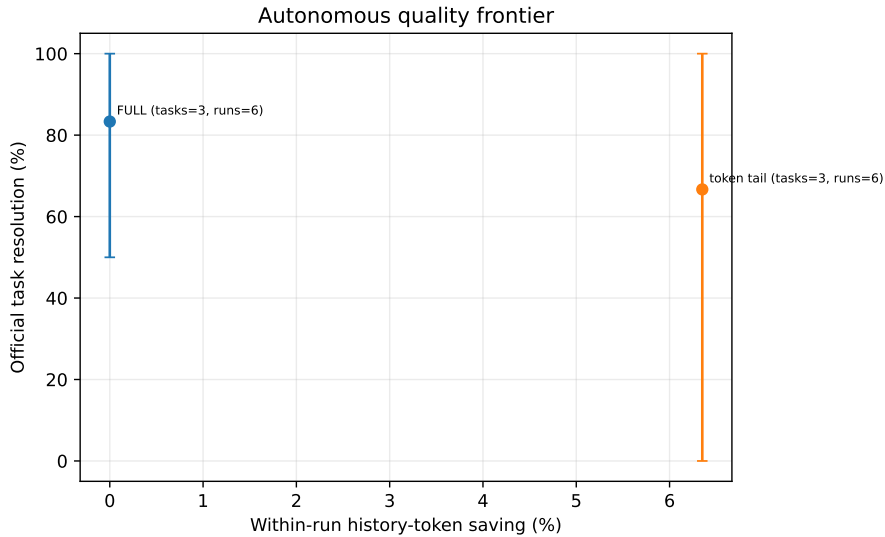


Figure 5: The first autonomous accuracy–saving point. Error bars are task-clustered and necessarily wide for three identities. The tail policy is not promoted because its point lies below the predeclared quality gate.

The Task 5 failure is mechanistically informative. Both treatment repeats are exact 19-call reproductions. They make the intended one-line source edit and inspect its unified diff, but the completion command emits a nearby source excerpt after the submission sentinel. The official harness therefore records **Patch Apply Failed**. The original task and submission instructions are still pinned, so this is not simple task-statement loss. A frozen terminal counterfactual also restores the complete history and reproduces exactly the same invalid command; no omitted terminal bundle can repair the already-formed path. The first repeat-qualified materialization effect occurs at

decision 3, where shortening a 5,757-token source observation to 5,468 tokens changes the exact reproduction command but preserves its diagnostic intent. Causal attribution of the later submission failure therefore requires a snapshot-restored rollout from that frontier, not terminal add-back. It also motivates a task-aware progress spine that protects mutation, verification, and submission state. The ungraded workspace is not counted as successful auxiliary evidence.

The campaign also tightened the evidence machinery. Exact pre-report patch failures and empty submissions are now definitive primary failures; a resident grader image is reused only after architecture inspection; and sparse request indices are rejected. One paused FULL solve has indices 1–11 and 13–22 and is preserved but excluded rather than silently renumbered. The raw runs, qualification decision, curves, and reductions are in `autonomous_nondominated_tail95_v1`.

11 Transfer

11.1 Second open-source harness

After policy qualification on mini-swe-agent, we transfer only the top policies to one richer harness such as SWE-agent or OpenHands. The goal is not a second exhaustive grid, but to test whether semantic-role findings generalize when the harness has richer tools and memory processing.

11.2 PRA Agent

The final transfer uses PRA Agent, where typed task, tool, result, progress, and subagent records are native objects. This tests whether a record-native agent can exploit the same policy more cleanly than a retrofitted linear-message harness.

12 Handoff to Native Runtime Evaluation

Paper 8.5 should freeze a small set of logical policies, for example

$$\{\text{FULL}, \text{BALANCED}, \text{AGGRESSIVE}\}.$$

A runtime study can then map the exact same `AgentMemoryPlan` to text replay, HF cache views, MLX interval views, vLLM pages, llama.cpp resident slots, and SGLang cache objects. The runtime study must not retune selection semantics per engine.

Paper 8.5 reports selected logical tokens, not K/V savings. K/V residence, copying, re-encoding, TTFT, and latency are measured only after the frozen plan is transferred to the runtime study.

13 Expected Outcomes

Our main hypothesis is

$$\text{token recency} < \text{causal recency} < \text{typed progress spine} + \text{retrieval}$$

at matched context budgets.

A positive result need not initially increase raw task success. A practically important result is same task success + fewer model calls + fewer rereads + fewer repeated tests + less retained history.

A negative result would also be informative: if simple recency performs as well as structured policies, coding-agent memory may be more local than assumed.

14 Limitations

Text realization recontextualizes selected records and therefore does not establish native-K/V equivalence. Frozen-trajectory screening also cannot replace autonomous evaluation. SWE-bench coding tasks may not represent research, browser, or multimodal agents. Semantic record labels may depend on heuristics during the first implementation.

These limitations are deliberate. This paper isolates logical memory selection before introducing engine-specific physical state. The current autonomous controls also show that fixed temperature and seed fields do not eliminate trajectory variation or submission failures in the tested endpoint; repeated FULL controls and separately labelled solution-state diagnostics are needed before attributing success or failure to a memory policy. Auxiliary workspace grades are post-hoc diagnostics and never replace the primary agent submission.

15 Conclusion

Agent memory selection is a distinct problem from transformer cache reuse. A serving engine may perfectly preserve selected native K/V while an agent policy selects the wrong history, and a good logical policy may be obscured by an incorrect engine implementation. We therefore study memory selection first, using an engine-independent record interface and a minimal software-engineering agent. The resulting policy frontier should provide a semantic contract for later native-memory runtimes: engines realize a frozen memory plan; they do not decide what the agent ought to remember. The first autonomous evidence further shows that this frontier cannot be estimated from a single control trajectory: the primary FULL submission resolves only one of two executions on each of two task identities, even though auxiliary grading shows both FULL-B workspaces contain resolving patches. The second task therefore fails the gate for heuristic evaluation. H3 remains negative under both primary and auxiliary outcomes on the first task. Within-record evidence selection provides a clearer but still non-inert frontier: a conservative structured materializer saves 5.54% on the repeatable 23-decision trace and preserves 54.5% of conservative actions, outperforming more aggressive positional materializers in agreement but not in compression. Observation receipts now fail closed when their own token cost is not smaller than the source and activate correctly on oversized mechanism controls. The evidence supports a sharper hypothesis than “semantic selection beats recency”: agent memory is not reducible to current world state. The new three-trajectory omission oracle nevertheless finds semantic-transition headroom of 24.5–38.2% under bounded subset search, while also showing that safe removals are non-additive and non-monotone. In the deployable frozen factorial, H1+H3 whole-record retirement misses its 2% mean yield gate and payload receipts are low-yield or harmful. A matched recency arm preserves all three semantic transitions at 3.0% mean saving and is the only candidate advanced to repeated autonomous qualification. In three same-host task identities it saves 6.35% of pooled treatment-history tokens, but primary resolution falls from 5/6 FULL runs to 4/6 treatment runs. Its 66.7% task-macro resolution misses the 80% gate. The repeated failure retains the intended source edit but submits a non-patch excerpt, motivating explicit progress/submission state rather than deeper blind recency. The structured observation arm supplies a more promising but still preliminary point: on one Task-5 execution it preserves official resolution in 22 calls while saving 6.10% of its trajectory-counterfactual materialized tokens; one of two FULL controls resolves, and the other creates a patch but submits an empty payload. It therefore advances to repeated multi-task confirmation, not to a profile. The persistent N=3 campaign then provides the first profile-quality discovery point. Blind R4 loses both paired Scikit-learn successes; R6 is unreliable; and R8 preserves success but saves only 13.09% while adding

calls. A frozen first-divergence oracle shows that this is selection-induced protocol conditioning, not merely backend variance, and motivates one clean completed action–observation exemplar per prior issue. The resulting R4/M2/V2/P1 arm resolves 6/6 issues across two executions, saves 36.41% and 45.82% against paired FULL, and reduces calls in both. This repeated discovery result conditionally qualifies the logical policy for frozen Paper 4.5 native-K/V transfer when equivalent episode metadata exists, but not yet for a general default: the uncertainty unit is still one ordered sequence family. In the realistic boundary-free setting, global R4 preserves all user instructions yet resolves only 2/3, adds 11 calls, and receives zero failure-aware saving. This rules out treating the explicit-boundary frontier as a deployable default. A two-level instruction-epoch policy fixes the immediate error by keeping the active epoch whole: its first admissible clean replication resolves 3/3 like persistent FULL, saves 34.83% on the paired denominator, and uses two fewer calls. This qualifies sequence expansion while retaining the stronger guardrail that dependent epochs need resource-DAG evidence. The N=6 expansion then rejects E0 as a deployment profile: it resolves 6/6 versus FULL’s 5/6 and removes 55.34% relative to its own trajectory, yet its 114 calls consume 43.87% more input tokens than the 60-call control. This is the paper’s clearest demonstration that per-request pruning can lose end-to-end. The completed E1/E2 ladder, which retains one or two recent instruction epochs whole, rejects both repairs: E1 resolves 5/6 but adds eight calls and reaches only 14.21% raw paired saving; E2 uses two fewer calls and reaches 21.93% raw paired saving but loses a FULL success, receiving zero failure-aware saving in that ladder. The frozen-prefix diagnosis reverses the outcome: E2 resolves 2/2 and FULL 0/2 at equal mean calls, with E2 sending 21.92% fewer tokens. Because byte-identical no-removal requests also diverge at temperature zero, this is evidence of a noisy outcome surface, not an accuracy gain. No instruction-epoch profile transfers to Paper 4.5 yet. The next valid test randomizes and interleaves repeated full-sequence E2 and FULL executions. The frozen profile screen identifies E0+M1/V1/P1 at 39.81% opportunity and R1/M1/V1/P1 at 37.43%, but their first autonomous pilots both fail with the same incorrect patch. Coarse semantic role floors therefore do not substitute for the two complete epochs that let E2 solve 2/2. The next causal test must compare E2 with the compact spine and restore one omitted causal group at a time before another autonomous profile advances. The one-to-ten-issue campaign continues to treat active-only retirement as session hygiene unless it beats deliberate fresh sessions on related or dependent sequences. Held-out sequences, additional models, and additional harnesses remain the confirmation frontier.

A Artifact Schema

Each request/run should persist:

- task ID and repository revision;
- mini-swe-agent revision;
- model/tokenizer revision;
- generation settings;
- canonical trajectory digest;
- typed record list and causal groups;
- selector name/revision;

- materializer name/revision;
- requested/realized budget;
- selected record IDs/spans;
- selected token count;
- next model response/action;
- primary autonomous submission result;
- separately labelled auxiliary final-workspace result and extraction availability;
- reread/retest/recovery counters;
- failure taxonomy labels.

B Initial Failure Taxonomy

MEMORY_TASK_LOSS
 MEMORY_SOURCE_LOSS
 MEMORY_MUTATION_LOSS
 MEMORY_VERIFICATION_LOSS
 MEMORY_PROGRESS_LOSS
 MEMORY_CAUSAL_PAIR_BREAK
 MEMORY_STALE_HYPOTHESIS
 MEMORY_REDUNDANT_REREAD
 MEMORY_REDUNDANT_TEST
 MEMORY_WRONG_FILE
 MEMORY_REPEAT_FAILED_EDIT
 MEMORY_FORMAT_REGRESSION
 MEMORY_RECOVERY_SUCCESS
 MEMORY_RECOVERY_FAILURE